- Python Review
- Data Handling
  - Variations on Classes
  - Malloc Trace
  - Memory Comparsion
  - Collection Module
  - Iteration
  - Builtin
  - Container Representation
  - Assignment
- Classes and Objects
  - Objects
  - Manipulating Instances
  - Static and Class Method
  - Class and Encapsulation
  - Inheritance
  - Special Method
  - Abstract Base Class
  - Advanced Inheritance
- Inside Python Objects
  - Underlying dictionary
  - Inheritance
  - Descriptor
  - Attribute Access Methods
- Functions
  - Function Arguments
  - Returning Value
  - Functional Programming
  - Closures
  - Errors
  - Testing and Debugging
- Working with Code
  - Variable Arguments
  - Scoping Rules

  - Function objects
  - Eval/Exec
  - Callable Object
- MetaProgramming
  - Decorators
  - Advanced Decorators
  - Class Decorator
  - Type
  - MetaClass
  - Application
- Iterators,Generators and Coroutines
  - Iterator
  - Producers & Consumers
  - Coroutine Functions
  - Ch8_Ex3 代码解释
  - Generator Control Flow
  - Managed Generators
  - 简单异步I/O服务器
  - Other Genarator
- Modules and Packages
  - Modules
  - Packages
  - __init__ File
  - Main Module

- **tag:**

Python Review

在之前Practical Python系列笔记中,我们只是简要介绍了一下数值型变量的运算符,在此我们介绍一下数值型变量的常用方法:

1. 浮点数的分数表示:as_integer_ratio

   ```
   x.as_integer_ratio()
   a, b = x.as_integer_ratio()
   print(a / b)    # 可还原为原始浮点数
   ```

   其返回一个二元组(p,q),表示该浮点数的精确有理数表示x=p/q.

2. 判断是否为整数:is_integer

   ```
   x.is_integer()
   ```

   is_integer()判断的是数值意义,与类型无关,这与我们之前常见的判断类型的不同,

   ```
   x = 4.0
   x.is_integer()      # True
   isinstance(x, int) # False
   ```

   is_integer是通过数值的角度判断是否为整数,而isinstance则是判断变量类型.

3. int的有理数视角:numerator/denominator

   ```
   y=12345
   y.numerator # 12345
   y.denominator # 1
   ```

   这是因为从数学角度来说,任意的整数本质上都是有理数.

4. 整数的二进制规模:bit_length

   ```
   y=12345
   y.bit_length
   ```

   他将会返回表示该整数所需的最少的二进制位数.其对应的数学计算公式为

$$BitLength(y) = \lfloor \log_2(y) \rfloor + 1$$

Data Handling

1 Variations on Classes

__slots__是一种节省内存的类定义方式.普通的python类的实例都有一个__dict__属性,用于动态存储属性.这为python类的使用带来了变量的灵活性,但是为了维护这个字典属性,导致每个实例的内存开销比较大.我们使用__slots__显式声明,这个类只允许有哪些属性,从而不再为每个实例创建__dict__属性,限制实例属性的动态增加,显著减少内存占用.

```
class Point:
    __slots__ = ('x', 'y')
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

其优点在于节省内存(适合存储大量的小对象),访问属性更快(少了一层字典查找);缺点在于不能随意添加新的属性,多继承上会报错multiple bases have instance lay-out conflict,这是因为对于多继承需要将两个父类的slot合并成一个实例,但是他不能保证生成一个安全的内存布局,slot在子类的实例是固定的内存偏移,两个独立父类都声明slot会可能出现重叠或冲突的情况.

dataclass是用于减少样板代码的类.很多类从本质上来说就是只是数据容器,却要重复写__init__函数,这是一些重复的工作.因此dataclass使用声明式方式来描述字段,由解释器自动生成常用方法.

```
from dataclass import dataclass
@dataclass
class Point:
    x:float
    y:float
```

这相当于就是手写了一个__init__的普通类.这个形式其实就是前面介绍过的装饰器,并且这个装饰器是可以传入参数的,常见的参数使用如下

```
@dataclass(frozen=True, slots=True)
class Point:
    x: float
    y: float
```

frozen=True表示实例不可变,slots=True表示自动生成__slots__属性.但是需要注意的是,dataclass里声明的类型并不是强制执行的,也就是你就算输入的类型和他声明的不同,他也不会报错,也会正常运行.

```
from dataclasses import dataclass
@dataclass
class Person:
    name: str
    age: int
p = Person("Alice", "30")  # age 其实是 str, 但 Python 不会报错
print(p)#Person(name='Alice', age='30')
```

其优点是极大减少了模板代码,可读性强,更接近数学/结构化定义,与类型注解天然结合;缺点在于不适合逻辑非常复杂,生命周期管理重的对象

namedtuple虽然其归入到类的变体,但其实他和元组会更接近一点,他是一个不可变的具名的元组.普通的元组内存小,不可变,而且只能依靠索引访问,可读性差.namedtuple在此基础上增加了字段名,在保留tuple行为的同时,提高了代码的可读性.他的生成有两种不同的方式

```
import typing
class Stock(typing.NamedTuple):
    name:str
    shares:int
    price:float
from collections import namedtuple
Stock = namedtuple('Stock',['name', 'shares', 'price'])
```

后者的第一个参数必须存在,他就是生成类的类名.他的最主要的特点就是不可变(线程安全,可以哈希化),内存占用小,以及支持解包,索引和迭代;他的缺点就是不能有可变字段,不适合复杂行为.

2 Malloc Trace

tracemalloc用于跟踪程序运行中的内存分配,一般用于帮助找出内存泄露和内存使用热点,在调试大型程序,排查内存增长十分有效.其只会跟踪Python堆内存分配,不会跟踪C扩展直接分配的内存.主要特点是他可以记录每次内存分配的位置(文件+行号),可以对比不同时间点的内存分配变化,可以按文件,行号统计内存占用.

启动跟踪

```
import tracemalloc
tracemalloc.start()
```

可选参数nframe:记录多少层调用堆栈,默认1.如果不是1,那么他就会检查当前函数以及(n-1)个函数栈内函数的内存分配.跟踪的内存是 Python 堆分配的对象.

获取当前内存快照

```
snapshot = tracemalloc.take_snapshot()
```

snapshot包含当前所有Python对象的内存分配信息,返回的是一个tracemalloc.Snapshotd对象,可以按文件,行号,函数等统计.基于此查看内存使用

```
top_stats = snapshot.statistics('lineno')  # 按行号统计
for stat in top_stats[:10]:  # 输出前 10 条
    print(stat)
```

里面的输出会有lineno表示代码的行号,size表示调用的内存大小,以及count为分配次数.对比两次快照,用于查看内存增长情况

```
snapshot1 = tracemalloc.take_snapshot()
# 执行一些操作
my_list = [i for i in range(10000)]
snapshot2 = tracemalloc.take_snapshot()
top_stats = snapshot2.compare_to(snapshot1, 'lineno')  # 比较内存变化
for stat in top_stats[:10]:
    print(stat)
```

其会输出内存分配增长最多的前十个的代码行.

查看当前内存总占用

```
print(tracemalloc.get_traced_memory())  # 返回（当前跟踪内存，峰值）
```

他返回的单位是字节,峰值可以用来帮助判断内存峰值的位置.

停止跟踪内存配置

```
tracemalloc.stop()
```

停止跟踪Python堆内存分配,停止跟踪后再调用快照会报错.

常用的工程场景为

1. 找内存泄漏:运行程序一段实践,对比内存快照,看哪些行内存不断增长
2. 分析热点代码:哪些函数/文件分配内存最多,优化数据结构或算法
3. 调试大对象:快速定位大数组,字典等对象的来源

snapshot.filter_traces(filters):过滤掉不关心的模块;Trace表示一次内存分配记录,而snapshot则是在某一时刻的所有trace的合集,这里面他会包含标准库,三方库以及自己代码的分配记录,通常这十分杂乱无章.这里的filter_trace会保留或排除某些trace,并且生成一个新的snapshot.这里的filters是一个TracebackFilter对象的列表:

```
tracemalloc.TracebackFilter(inclusive: bool,filename_pattern: str)
```

这里的inclusive的作用十分关键,True表示只保留匹配该模式的trace,False则是排除匹配该模式的trace.filename_pattern则是一种glob风格的路径匹配字符串,这个glob风格的主要特征其实就是我们前面提到的模糊查找,用通配符匹配字符串,常用的例子为

```
'*'
"*/site-packages/*"
"*/lib/python3.11/*"
"*/your_project/*"
```

例如我们将标准库和三方库的内存分配舍弃,只看自己的代码分配

```
import tracemalloc
snapshot = tracemalloc.take_snapshot()
filters = [
    #过滤 Python 导入系统内部产生的内存分配记录
    tracemalloc.TracebackFilter(False, "<frozen importlib._bootstrap>"),
    #过滤 tracemalloc 无法追溯到 Python 源文件的内存分配
    tracemalloc.TracebackFilter(False, "<unknown>"),
    tracemalloc.TracebackFilter(False, "*/lib/python*/*"),
    tracemalloc.TracebackFilter(False, "*/site-packages/*"),
]
snapshot = snapshot.filter_traces(filters)
```

只看某个模块/文件夹的内存

```
snapshot = snapshot.filter_traces([
    tracemalloc.TracebackFilter(True, "*/myproject/*")
])
```

snapshot中只包含myproject目录下产生的内存分配.

3 Memory Comparsion

我们用不同的方式读取同一个csv文件并且用tracemalloc测量了内存使用情况.为了简便起见,我们不贴完整的代码,只是做简要的结果分析.

首先,用如下的代码来将文件直接读取成一个字符串,

```
data=f.read()
```

内存消耗约12.36MB,只读取原始字符串,没有任何解析,也没有创建额外对象,这是最为节省内存的方式,仅需要存储文本内容,这样只能做字符串处理,不需要做结构化访问.

用如下的代码来将文件按行读取成字符串列表,

```
data=f.readlines()
```

内存约为40.73MB,这一过程创建了约57万条str对象和一个存储这些对象的列表.但我们发现他的内存显著高于前一个方法,这是因为Python的字符串对象有额外的开销,因为每个str都是一个对象,包含元数据,因此他比直接存储需要消耗更多的内存空间.

用如下的代码来将文件读取成元组列表,

```
records.append((row[0], row[1], row[2], int(row[3])))
```

内存约为114.73MB,这个时候每行已经从一个字符串对象变成了4个元素的tuple,每个元组中有三个字符串对象和一个整数对象,每个tuple仍然是独立对象.因此他每一行其实被存储在1个tuple对象,3个字符串对象和1个整数对象中,故而其会比读取字符串列表有明显的内存消耗提升.

用如下的代码来看将文件读取成一个字典列表,

```
records.append({headings[0]:row[0],headings[1]:row[1],headings[2]:row[2]
,headings[3]:int(row[3])})
```

内存约179.42MB.字典是最为占用内存的常见容器之一,每个字典都有自己的哈希表,即使只有四个key也会在分配空间时预留空间,稀疏结构存储.key是字符串,value是字符串+int.因此他的内存效率低但是他的访问十分灵活

用如下的代码做一个自定义类列表读取,

```
class Row:
    def __init__(self, route, date, daytype, rides):
        self.route = route
        self.date = date
        self.daytype = daytype
        self.rides = rides
for row in rows:
    records.append(Row(row[0],row[1],row[2],int(row[3])))
```

内存约128.59MB.类存储虽然也是使用的__dict__来做存储本质上也是一个字典,但是由于每个实例字段名完全一样且内存分布一致,因此Python会自动复用相同的字符串和整数对象,故而总对象数更少,间接的降低内存.dict为速度牺牲空间,类在值复用且结构一致下更省.

用如下代码将文件用namedtuple的方式存储,

```
Row=namedtuple('Row',['route','date','daytype','rides'])
```

内存约为119.36MB.namedtuple本质上是tuple的子类,但支持使用字段名访问.由于他的底层实现还是基于元组完成的,因此他的内存比普通的class更省.而且每个实例都没有__dict__字典,属性通过索引访问,开销接近普通tuple.

用如下的代码将文件用使用__slots__的类

```
class Rows:
    __slots__ = ['route', 'date', 'daytype', 'rides']
```

内存约110.11MB.__slots__禁用了__dict__属性,将实例属性直接存储在固定数组里面,减少了内存碎片和对象开销.

4 Collection Module

defaultdict主要是用来做一对多映射问题的,Counter则是一个计数器,并且他提供了排序的功能.我们将两个对象组合在同一个代码中简要介绍一下,

```
dict_year_routes=defaultdict(Counter)
for r in rows:
    year=r['date'].split('/')[2]
    dict_year_routes[year][r['route']]+=r['rides']
    differ=dict_year_routes['2011']-dict_year_routes['2001']
    print(differ.most_common(5))
```

这里的代码创建了一个默认类型为Counter的defaultdict类.我们通过dict_year_routes[name]的方式访问的是一个Counter对象,其内部仍有一个字典形式,因此需要再做一次索引才可以到达计数的功能.最后的differ是两个计数器类的作差,其实就是其中每个键的计数作差,但是只保留结果为正数.

```
from collections import Counter
c1 = Counter(a=5, b=3, c=2)
c2 = Counter(a=2, b=4, d=1)
result = c1 - c2
# 等价于:
# a: 5 - 2 = 3 → 保留
# b: 3 - 4 = -1 → 舍弃 (不保留 ≤ 0 的值)
# c: 2 - 0 = 2 → 保留
# d: 0 - 1 = -1 → 舍弃
```

deque则是一个双端队列,因此他主要是处理队列相关的问题,例如他可以用来存储最后N个对象的历史.

```
history=deque(maxlen=3)
with open('../Data/ctabus.csv','r') as f:
    for line in f:
        history.append(line.strip())
        print('Last 3 lines in the file:')
for record in history:
    print(record)
```

ChainMap用于将多个字典组合成一个单一的,可更新的视图,而不需要实际的合并他们.在需要按优先级顺序多个作用域(如配置,环境变量,默认值)时,传统做法是层层嵌套

```
if key in dict
```

或者手动合并字典.但这样的话,合并会复制已有的数据,浪费内存;无法动态反应底层字典的变化;出现了逻辑冗余.ChainMap提供了一个轻量级的视图,按顺序搜索键,并且支持写操作,默认写入第一个映射.

```
from collections import ChainMap
defaults = {'color': 'red', 'user': 'guest'}
env_vars = {'user': 'alice'}
cli_args = {'color': 'blue'}
# 组合：优先级从高到低(cli_args>env_vars>defaults)
config = ChainMap(cli_args, env_vars, defaults)
print(config['user'])
print(config['color'])
config['debug'] = True
print(cli_args)
```

这里的debug键对会直接写入cli_args,也就是第一个字典映射.ChainMap的主要特性是

1. 查找顺序:从左到右搜索,第一个匹配的键生效
2. 写/删操作:默认作用在第一个映射(如果想要修改可以通过子类重写的方式)
3. 动态性: 底层的字典变化可以直接反映到ChainMap里面
4. 不复制数据:内存开销极小(仅存储对原字典的引用)

常用的方法是.maps返回底层字典列表(可以进行修改,其修改会直接影响到ChainMap的结果);.new_child()在底层字典列表前面添加一个新的空字典,常用于作用域压栈,他会返回一个新的ChainMap,不是对原有ChainMap的就地操作;.parents返回除第一个外的所有映射,类似弹出顶层作用域.

这里需要注意的是ChainMap不是字典的替代品,而是多字典联合查找的工具.如果你需要合并所有的字典键对,那你需要做一些手动合并操作,对于某些只读场景,其实Python3.8以上提供的dict|dict操作符可能也是不错的选择,但是他并不支持动态更新.

```python
d1 = {'a': 1, 'b': 2}
d2 = {'b': 3, 'c': 4}
merged = d1 | d2
print(merged)  # {'a': 1, 'b': 3, 'c': 4}
# 上面体现出了合并的时候相同变量名的右边优先级更高
# Python3.9以上则提供了就地操作符
merged|=d1
print(merged)
```

5 Iteration

在遍历一个可迭代对象时,如果我们只关心循环次数或者某种元素的变化,而对对象里的其他部分不感兴趣,那我们可以用下划线作为占位符来表示我并不关心这个数值.

```python
for _ in range(5):
    print('Hello')
```

其实下划线是一个合理的变量名,只是按照通用的惯例而言,我们会默认这个值是被抛弃的.但是他仍然会占用内存,只是提醒程序员这个值并不会被使用.

通配符解包:Python中引入了*用于解包多个元素,常用于抓取剩余项.

```python
a,*rest,b=[1,2,3,4,5]
print(a,rest,b)# 1 [2,3,4] 5
```

中间的解包元素会捕获中间所有没有显式分配的元素.这样的解包只能出现一次.当然我们也可以与上面的下划线合并形成的丢弃不需要的没有显式分配的元素.

```python
first,*_,end=[1,2,3,4,5]
print(first,end) # 1 5
```

解包可迭代对象,使用*或直接赋值来将可迭代对象(如列表,元组)的元素分配给多个变量.

```python
x,y,z=[1,2,3]
def f(a, b, c):
    return a + b + c
args = [1, 2, 3]
f(*args)  # 等价于 f(1, 2, 3)
```

他比我们常用的+更好一点,因为+要求操作数同类型且支持拼接的序列(如list+list),如果用的如list+tuple的情况,那么如果用加号就会报错,但可以用解包符来提取元素.

解包字典,使用**来将字典解包为关键字参数

```
def greet(name, age):
    print(f"Hello {name}, you are {age} years old.")

info = {'name': 'Alice', 'age': 30}
greet(**info)  # 等价于 greet(name='Alice', age=30)
```

解包符也可以应用在函数参数里面,因为Python的函数只需要满足位置参数放在关键字参数(默认参数)之前,各种参数类型的组合就是合法的.

```
def func(pos1,pos2,...,default1=..,default2=...,)
a=(1,2,3)
b={'c':3,'d'=4}
func(1,*a,**b,flag=1)
```

生成器是一种惰性求值迭代器,他在遍历的时候逐个产生值,不会保存已生成的值.一旦遍历完成就不能够从头开始.

```
gen = (x**2 for x in range(3))  # 生成器表达式
list(gen)  # [0, 1, 4]
list(gen)  # [] — 已耗尽，无法再次使用
```

如果我们需要多次使用的话就应该将其转换为列表或者重新创建生成器.这是因为生成器内部维护一个状态(当前执行到哪一步),一旦结束(抛出StopIteration),状态就终止了.

6 Builtin

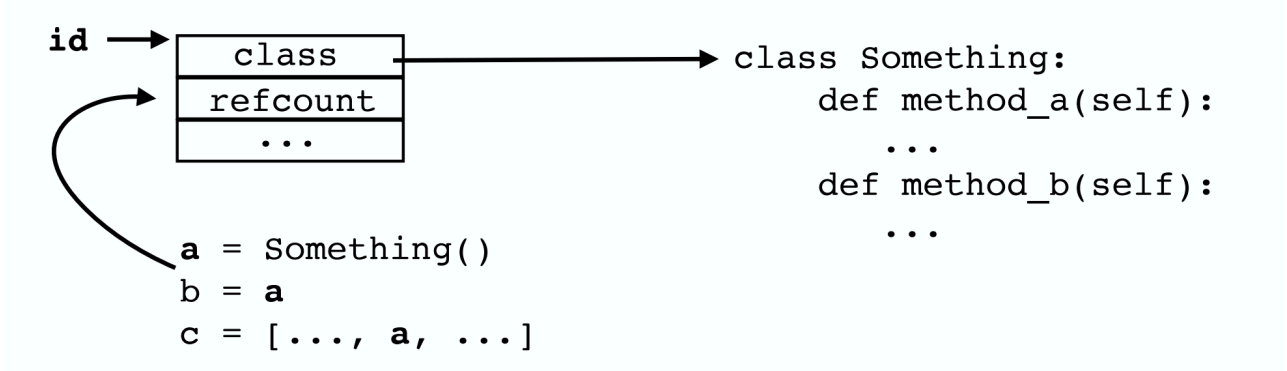
内置类型(Builtin types)是Python解释器原生支持的对象类型,如int,float等.他们几乎都是基于C语言编写的,直接写入Cpython解释器内部.这些内置类型是最为基础的,最为高效的对象类型.换言之,如果我们声明一个内置类型对象,其实是创建了一个由C结构体表示的,高度优化的底层对象.

无论是否为内置类型,每个Python对象在内存中都包含三个基本字段:

1. id为对象的唯一标识,实际上是他在内存中的逻辑地址,利用id(object)读取.
2. class为对象的类型,利用type(object)读取,决定了它有哪些方法和属性
3. refcount为引用计数,用于垃圾回收(当计数归零时,对象会被释放)

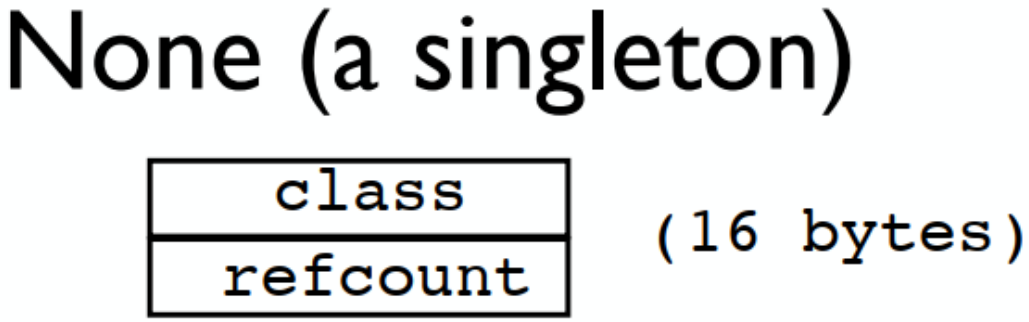
```
a=[1,2]
b=a # refcount+=1
c=[a,None] # refcount+=1 again
```

上面的这个结构是对所有对象都是通用的,但是内置类型的字段会更加紧凑,访问速度更快,因为他的底层设计是C语言的结构体,而不是动态字典.



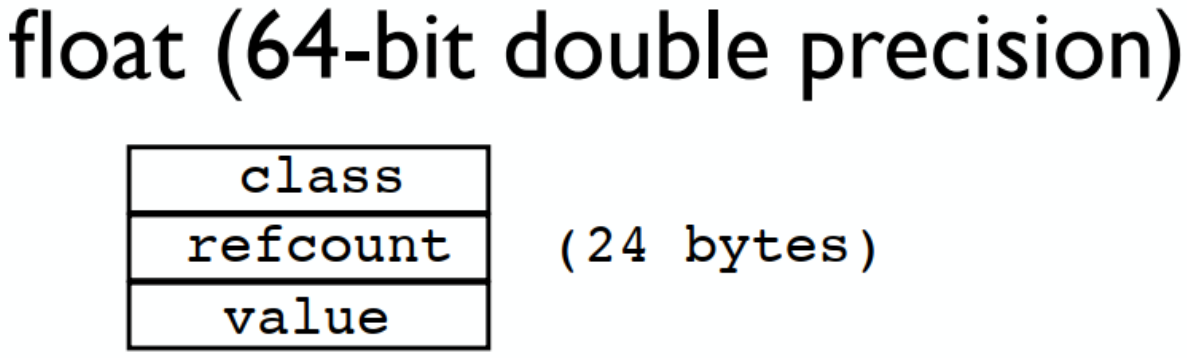
简要介绍一下内置类型的内部表示和内存开销:

1. None是一个单例对象,全局只有一个None实例.内存结构也是十分简单的,没有数据字段,只有对象头.因此他的内存占用极小,但仍然有基础的内存开销.



2. float是64位双精度浮点数类型,其内存总大小是24字节(在64位系统上).其中class指针占用8字节,refcount占用8字节,value也会占用8字节.其验证代码如下:

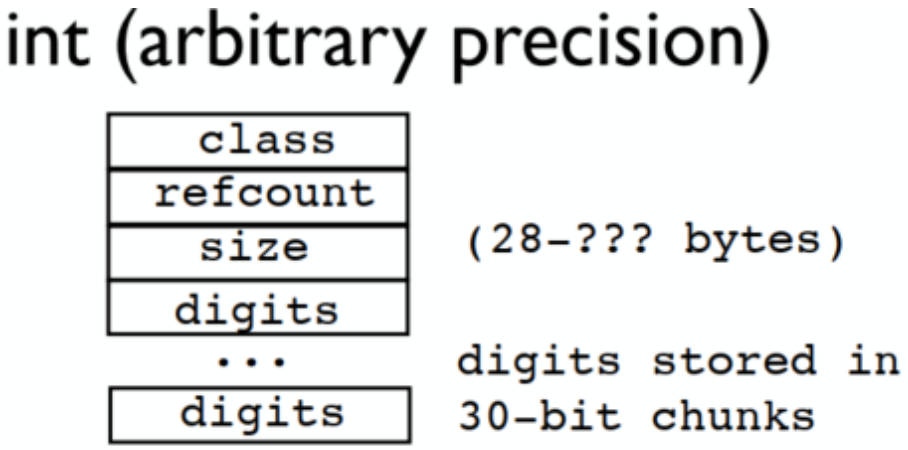
```
import sys
print(sys.getsizeof(2.3))
```



3. int类型表示任意精度整数,他并非是C/C++那样的固定内存32位或64位.其中class指针占用8字节,refcount占用8字节,size表示有多少个digit块,占用8个字节,真正的存储位置在digit列表里面,每个元素都是4个字节.但是存储数据的32位中我们需要提供两个位次来做进位或上下溢判定,因此我们只有30位用于存储数据.其验证代码如下:

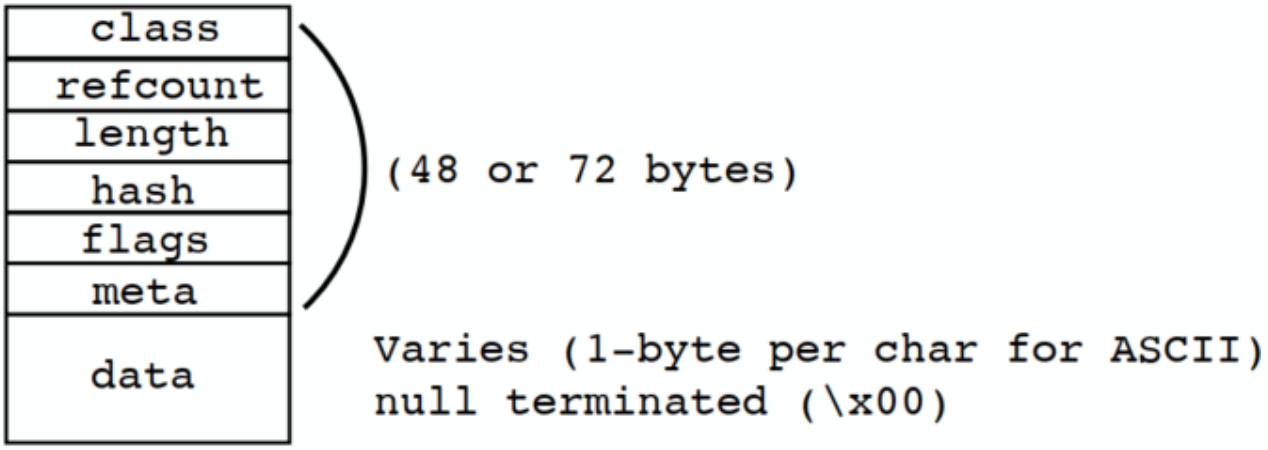
```
import sys
print(sys.getsizeof(0)) #28
print(sys.getsizeof(2**30)) #32
print(sys.getsizeof(2**60)) #36
print(sys.getsizeof(2**90)) #40
```

因此,从上面的运行看,python内置的int类型看,会随着数值的增大,digits数组变长,从而导致内存线性增长.



1. str则是python的字符串类型.我们主要介绍一些独有的字段属性.length存储的是字符串中字符的数量,并不是字节数,这样可以使得我们在查询长度的时候时间复杂度降为O(1),对于大量查询字符串长度的程序具有极高的优化.hash属性卡会存储字符串的hash值,因为对于Python而言,字符串一般是用作字典的键或集合的元素.这些数据结构需要依赖对象的哈希值来实现:快速查找,去重和等价判断.他可以存储hash值是因为字符串是一个不可变对象,故而生成一次hash值,他的hash值就不会有任何变化.她会在首次调用或用作dict/set的键时计算,Python会计算hash值并存储以后的调用都是O(1).flags是一种标志位,用于记录字符串的编码格式和状态,用于实现自适应字符串表示.如ASCII表示一个字符一个字节,Latin-1表示一个字符两个字节,USC-2/USC-4表示一个字符两个或四个字符.data则是实际的字符数据,他可以按需选择字符宽度,并以\x00结尾,便于和C函数交互.

```
import sys
print(sys.getsizeof('n')) # 1 ASCII char → 1B data + overhea
print(sys.getsizeof('ñ')) # 1 Latin-1 char → 2B data + overhead = 74B
```



Python对象有显著的固定内存开销.频繁创建小对象(如解析大量文本生成无数短字符串)会导致高内存消耗.

内置类型通过设置对象协议来实现某些功能.Python的运算符和内置函数是通过特殊方法实现的,这些方法构成协议.

```
a=2
b=3
print(a+b) # 相等于a.__add__(b)
print('hello') # 等价于"hello".__len__()
```

协议是解释器的硬编码行为.字节码直接调用协议方法,可以用dis模块查看编译后的字节码,如下所示

```
import dis
def bar(x,y):
    return x+y
print(dis.dis(bar)
      '''
 25           0 RESUME                   0

 26           2 LOAD_FAST                0 (x)
              4 LOAD_FAST                1 (y)
              6 BINARY_OP                0 (+)
             10 RETURN_VALUE
None
'''
```

RESUME 0是Python3.11引入的一个帧初始化指令,用于职场更快快的函数调用和生成器恢复.他对协议机制并不会有什么太大的影响.LOAD_FAST是将局部变量压入求值栈.BINARY_OP是Python3.11的通用二元操作指令,他会触发对象的__add__协议.

我们可以自定义一些自己的内置类型,并且实现一些协议,让自定义内置类型可以表现十分自然.

```python
from functools import total_ordering

@total_ordering
class MutInt:
    __slots__=['value']
    # Mutable Integers
    def __init__(self,value=0):
        self.value=value
        # Fixing output
    def __str__(self):
        return str(self.value)
    # !r的作用是在f-string中调用repr()
    # !s的作用是在f-string中调用str()
    # !a的作用是在f-string中调用ascii()
    def __repr__(self):
        return f'MutInt({self.value!r})'
    # 用于f-string,string.format()做格式化自动调用
    def __format__(self,fmt):
        return format(self.value,fmt)
    # Math Operator
    def __add__(self,other):
        if isinstance(other,MutInt):
            return MutInt(self.value+other.value)
        if isinstance(other,int):
            return MutInt(self.value+other)
        else:
            raise NotImplemented

    __radd__=__add__# Reversed operands

    def __iadd__(self,other):
        if isinstance(other,int):
            self.value+=other
            return self
        if isinstance(other,MutInt):
            self.value+=other.value
            return self
        else:
            raise NotImplemented
        # Comparisons
    def __eq__(self,other):
        if isinstance(other,int):
            return self.value==other
        if isinstance(other,MutInt):
```

```
            return self.value==other.value
        else:
            raise NotImplemented
    def __lt__(self,other):
        if isinstance(other,int):
            return self.value<other
        if isinstance(other,MutInt):
            return self.value<other.value
        else:
            raise NotImplemented
        # Coversions
    def __int__(self):
        return self.value
    def __float__(self):
        return float(self.value)
    __index__=__int__
```

我们需要解释一下上面的代码.__format__函数是用于f-string,string.format()做格式化自动调用.__add__函数只能够支持作为加法的左操作数,为了满足加法交换律,我们重新定义了__radd__函数,程序如果没有找到合适的加法函数,会自动调用radd函数.但是这样只能够覆盖加法的操作,如果我们希望考虑加法的就地操作,那么需要实现__idd__函数.对于逻辑对比关系,我们需要实现如__eq__函数来判断相等,__lt__函数来判断小于.但是如果我们对每个逻辑判断关系都去写,那会是十分冗余的,因此引入functools模块的total_ordering装饰器,他可以让我们只实现一个关系比较,从而自动补全其余的.我们同样可以为自定义内置类型可做一个类型转换函数,但是需要注意的是,如果只是设置了类型转换函数,他并不能在索引中自动转换为合适类型,需要手动操作,但可以重定义__index__函数.

7 Container Representation

和C/C++不同的是,Python的容器只会存储值的引用,而不是值本身.因此容器的操作实际上也是操作引用,而不是操作对象.对于Python中的可变容器而言,如list,dict和set,他不是按照需要分配恰好的内存,而是会过度分配(over-allocation),这是为了方便append和insertion操作加速.如果我们提前分配了额外的空间,那么append操作会十分快,因为空间已经准备好了,不需要重新向系统请求分配空间.

容器内存的增长行为:容器的使用内存的增长会是现使用内存的某种比例.

1. List的内存增长方式是当内存占用满了,会增加现有内存的约12.5%,并非固定倍数.其测试代码如下所示,

```
a=[]
print(sys.getsizeof(a))
   for i in range(10):
       a.append(i)
       print(a,sys.getsizeof(a))
       # empty 56; [0,1,2,3] 88
       # [0:4] 120; [0:9] 184
```

空列表会消耗56个字节,如果加入一个数字会消耗88个字节,此时开辟了32个字节的空间,相当于四个位置;一直等到存储完四个位置,开始存储第五个位置的时候,列表会请求增加内存空间,将88个字节扩充到120个字节,同样允许继续存储四个位置,一直到存储到第八个位置,开始存储第九个位置的时候,列表会继续请求增加内存空间,将120扩充到164个字节,也就是允许继续存储8个字节.这里我们需要声明的是,引用需要占用8个字节是因为所使用的电脑是64位的,如果用的32位系统则是4个字节.

1. Set的内存增长方式是当存储元素占用了开辟空间的2/3,就会将开辟空间增长4倍.但是电脑上测试出来,他的增加阈值应该是3/5,具体可能需要查阅python相应版本的底层设计.

```
a=set()
print(sys.getsizeof(a))
   for i in range(20):
       a.add(i)
       print(a,sys.getsizeof(a))
       # empty 216; {0,1,2,3} 216
       # {0,...,4} 728; {0,...,18} 2264
```

2. Dict的内存增长方式是当存储元素占用了开辟空间的2/3,就会将开辟空间增长2倍.

```
a={}
print(sys.getsizeof(a))
for i in range(20):
    a[i]=i+1
    print(a,sys.getsizeof(a))
    # 第6个元素扩容 第11个元素扩容
```

这里的逻辑是初始是8个空间,其空间的2/3并向下取整是5,也就是当存储空间使用了5个元素,添加第6个元素的时候需要重新开辟空间,其增长为两倍,所以会开辟至16个空间,其空间的2/3并向下取整是10,因此第11个元素才会重新开辟新的空间.

字典和集合的使用中会经常需要做键值查找,因此他们的底层实现都是基于Hash表完成的,这是因为Hash表的查找是O(1)时间复杂度的.因此我们需要介绍一下关于集合和字典的哈希化.字典和集合的键通过使用__hash__函数获得其hash值,并存储起来,在程序内部使用.因此这要求字典和集合的键值必须是可以哈希化的对象,也就是字符串,数字或者元组.而列表,集合和字典等不能被用于哈希化,故而不能成为字典和集合的键值.如果我们需要自定义一个数据类型,

并希望他可以成为字典或集合的键值,我们需要实现__hash__()方法和__eq__()方法,其中hash方法是为了保证类型可以哈希化,而eq方法则是保证相同数值的哈希值一样,虽然不会妨碍使用,但是如果没有他那么可能会出现同一个值有不同的哈希值,这与哈希化的初衷相悖.

在此简要介绍一下字典的内部布局.我们向程序输入一个字典, 系统会计算字典键值的hash值,并依据这些hash值将entiries数组的引用存储到hash表的相应位置上,entries数组会存储相应的键值对.如果我们要查找某个键值对,其实就是上面过程的逆向,因此不在此赘述.这里其实就会出现一个很自然的问题,也就是哈希冲突问题,当hash表不断被填充的时候,发生原有的位置被占据了的情况是十分常见的,因此我们需要一个解决冲突的算法来除了这个问题.Python中引入了一个扰动寻址法的方式来处理哈希冲突问题,其代码如下,

```
# 初始位置
mask = table_size - 1
i = hash(key) & mask
# 冲突后迭代探测
perturb = hash(key);  # perturb 初始为完整哈希值
do {
    i = (i * 5 + perturb + 1) & mask
    PERTURB_SHIFT = 5
    perturb >>= PERTURB_SHIFT
} while (冲突);
```

这个算法结合了线性项(i*5+1)提供基础的伪随机探测序列,并加上了扰动项perturb,将Hash值的高位引入探测过程.我们常见的探测序列其实并没有扰动项,添加扰动项则是可以解决低位聚集问题.哈希表索引通常使用的hash&mask计算,他只使用哈希值的低位.当多个键的低位相同的时候,尽管高位可能有很大的差异,他们还是会聚集在表的某个区域,导致线性探测容易继续陷入局部聚集,并重复访问已知冲突区域,降低匹配效率.扰动策略是逐步将哈希值的高位引入探测序列,这样探测序列会受哈希值的每一位,极大降低冲突的概率.

这里我们发现他的求余运算是通过按位取和运算实现的,这是因为我们一般设定哈希表的长度为2的幂次.对于任意一个整数,我们都可以将其写成一个唯一的二进制形式,

$$h = \sum_{k=0}^{m} a_k 2^k = \sum_{k=0}^{n-1} a_k 2^k + \sum_{i=n}^{m} a_i 2^i$$

因此,h对哈希表长度2^n取余得到的结果其实就是最后n-1位二进制的值.而如果我们考虑的是与哈希表长度减一按位取和操作,因为哈希表的长度为2^n,那么长度减一的二进制形式其实就是最后n-1个二进制位为1,其余位置为0,按位取和其实得到的结果就是原有整数的二进制的后n-1位二进制的值,这与取余操作是一样的,因此我们可以将两个运算替换使用.

和上一小节一样,我们也可以自定义容器类,只需要设置好必须的协议方法.如果我们需要创建一些新的容器,我们可以使用collections.abc中定义的抽象基类.常用的抽象基类如下所示,

抽象基类	必须实现的方法
Container	contains(用于支持in操作)
Iterable	iter(支持for循环)
Iterator	iter,next(自定义迭代器)
Sequence	getitem,len
MutableSequence	getitem,len,setitem,delitem,insert
Mapping	getitem,iter,len
MutableMapping	getitem,iter,len,setitem,delitem
Set	contains,iter,len
MutableSet	contains,iter,len,add,discard

8 Assignment

Python的许多操作很多都是基于赋值操作的.与C/C++不同的是,Python的赋值操作并不会复制被赋值的值的副本,因此所有赋值都只是复制被赋值的值的引用.所以赋值操作会导致多数变量指向同一个元素.我们修改其中一个变量,会使得全部变量的数值都发生修改,因此在修改变量的时候需要谨慎检查引用.同样函数调用参数其实也是以引用的形式传入,而不是传入值,所以参数在函数内部的变化是可以影响到外部的.需要注意的是,Python中许多对象类型是不可变的.

虽然上面提出Python的赋值操作是赋值引用副本,但如果我们对他进行重赋值并不会影响原有的引用赋值副本,而是指向了另一个存储空间.is操作符可以用于检测两个变量是否指向同一个地址.

```
a=[1,2,3]
b=a
print(a is b)
```

而所有的对象都是具有自己独特的整数标识码,这种标识码可以用id函数读取.

```
print(id(a))
print(id(b))
```

上面我们介绍了Python赋值操作的一些易错点,因此我们简要介绍一下不可变对象的优点.不可变对象可以安全地共享数据并不用担心数值被修改;由于其不可变性,共享可以节省相当多的内存.

容器中提供了一些方法来做复制操作,如

```
a=[2,3,[100,101],4]
b=list(a) # make a copy
print(a is b) #False
```

然而,他只保证了外层容器的独立性,对于内层容器,依旧是指向相同的地址.

```
# shallow copies
a=[2,3,[100,101],4]
b=list(a)
print(a is b) # false
print(a[1] is b[1]) # false
print(a[2] is b[2]) # true
```

如果我们需要完全复制某个对象以及其中的所有元素,那么可以选择使用copy模块的deepcopy函数.这也是唯一安全的方式来复制变量.

Classes and Objects

1 Objects

面向对象编程其实就像是基于行为的自下而上的建模.一个对象则会包含一些内部状态和一些对内部状态的操作.因此,对于一个对象而言,其中包含的数据和行为是密不可分的,数据是对象的属性,而行为则是体现了对象的特性.

Python中一般用class声明语句来定义一个新的对象.

```
class Player:
    def __init__(self,x,y):
        self.x=x
        self.y=y
        self.health=100
    def move(self,dx,dy):
        self.x+=dx
        self.y+=dy
    def damage(self,pts):
        self.health-=pts
```

从上面我们可以看出类其实是一系列的函数,从而提供对对象示例的不同的操作方式.这里我们需要强调的是,class声明语句只是提供了类的定义方式,并不提供一个可以操作的对象.在程序中,我们需要对类做实例化得到对象示例,才可以调用方法进行相关操作.

每个实例都具有他自己的局部数据,这个数据是每个实例单独占有的,其他的实例不能随意访问.实例数据的初始化一般是基于__init__函数,换言之任意存储在self对象的数值都是实例数据.对于实例属性的数量或类型没有任何限制.实例方法则是指应用在对象实例的函数,一般都是在类声明语句中已经给出了相关定义.这里需要注意的是实例方法的第一个参数一般都是对

象.按惯例而言,实例一般会记为self,但这个只是一个习惯,并不是很重要,可以用其他的名字来代替.实例方法其实就是普通函数的定义,只是约定俗称将实例对象作为第一个参数 传入.

对象的属性指的是通过.来访问的.例如

```
a.x # 实例属性
Player.move # 类属性
import math
math.pi # 模块属性
```

在Python中类本身并不会创建一个新的作用域,因此在类的方法内部调用其他方法时,必须显式地通过self来引用实例方法,否则Python会去全局作用域中查找同名函数,从而可能导致NameError或调用错误的函数.在Python中,作用域规则遵循LEGB(Local-Enclosing-Global-Builtin).类的代码块不是一个封闭作用域,类中定义的方法是独立的函数,他的局部作用域只包含自己的参数和内部变量,方法内部无法直接调用类中定义的其他方法或属性,除非通过self(实例)来访问.

2 Manipulating Instances

对实例属性的操作一般是三个形式:

```
obj.attr # Get an attribute
obj.attr=value # Set an attribute
del obj.attr # Delete an attribute
```

一旦实例被程序创建,那么对于实例的属性可以自由的添加和删除.对实例属性的操作,存在如下的几个属性访问函数,

函数	作用
getattr(obj,'name')	obj.name
setattr(obj,'name',value)	obj.name=value
delattr(obj,'name')	del obj.name
hasattr(obj,'name')	Tests if attribute exists

这里我们需要注意的是getattr()具有一个比较常用的默认参数值,

```
x=getattr(obj,'x',None)
```

如果属性不存在,返回的默认值.如果不提供且属性不存在,会抛出AttributeError.

前面我们提到了实例的属性不止有数据还有方法,因此我们需要简要介绍一下实例方法的调用.方法的调用其实是分为两步的,第一步需要先通过.操作符查找到实例的方法对象,第二步则是通过()操作符来调用方法.

```
s=Stock('ACME',50,91.1)
c=s.cost # 查找并返回方法对象
print(c()) # 调用方法
```

当我们通过一个实例访问他的方法,例如obj.method,即使尚未调用,这个方法已经是一个bound method了,因为他已经绑定了该实例作为self.bound method只出现在实例访问方法,如果是通过类名访问的话,那么获得的对象是普通的函数对象,而不是绑定方法.绑定方法可以通过__self__属性来访问绑定的实例对象属性,通过__func__属性来访问绑定方法的函数属性.因此从这两个角度来说,我出门可以得知调用绑定函数的方法其实就是通过上面的属性完成的,

```
c.__func__(c.__self__)
```

3 Static and Class Method

前面我们提到了类的定义,它包含了类的实例化对象所共同使用的属性定义.只需要在类定义中定义一次,就可以在所有实例中使用,他会在实例创建的时候被调用,从而为实例创建属性.类属性指的是在__init__函数中定义的实例属性以外定义的变量,可以通过类名调用类变量,同时也可以通过实例名调用类变量.这里需要注意的是,类变量是可以被所有的实例调用,并且对于类变量的修改有如下几种情况,

```
class Dogs:
    species = "Canis familiaris"
    owner=[]
d1=Dogs()
d2=Dogs()
d1.species='Wolf' # 创建同名的实例属性
print(d1.species) # 返回Wolf
print(d2.species) # 返回类属性
print(Dogs.species) # 返回类属性
Dogs.species='Wolf' # 可以修改类属性
d1.owner.append("John") # 可变类变量,可以通过实例修改内容并影响所有实例
print(d1.owner)
print(d2.owner)
print(Dogs.owner)
```

在这里我们需要额外补一句,如果类中设置了__str__方法,那么print函数会自动调用类中定义的__str__方法,而如果str方法没有被定义,那么会查找类是否定义了__repr__方法,如果也没有,那么就会返回<__main__.ClassName object>.除了上面提到的一些修改类变量的方法,我们也可以通过类继承的方式来修改类变量.

除了类变量以外,还可以对于类定义类方法.类方法是直接操作类自身,与普通的函数不同的是,类方法需要用@classmethod装饰器定义.

```
class SomeClass:
    debug=False
    def __init__(self,x):
        self.x=x
        @classmethod
    def yow(cls):
        print('SomeClass.yow',cls)
```

这里和实例方法类似,类方法中类名会以第一个参数的形式传入.类方法的主要作用其实是可以定义一些不同于__init__函数的初始化函数,如

```
class Date:
    datefmt='{year}-{month}-{day}'
    def __init__(self,year,month,day):
        self.year=year
        self.month=month
        self.day=day
    def __str__(self):
        return
self.datefmt.format(year=self.year,month=self.month,day=self.day)
    @classmethod
    def today(cls):
        tm=time.localtime()
        return cls(tm.tm_year,tm.tm_mon,tm.tm_mday)
```

类方法也可以在一些类继承的情况下解决较为复杂的问题.

除了类变量和类方法以外,类的定义域中还可以存在静态方法的定义.静态方法虽然也是类定义的一部分,但是他其实不会对类或实例进行操作.如果在类中定义了两个同名的函数,那么后面定义的函数定义会覆盖先前函数的定义,这是因为类会创建一个临时的命名空间,只能存在一个同名对象,因此同名标识符的后续赋值会覆盖先前绑定的对象.与前面提到的实例方法和类方法不同的是,静态方法不存在暗含的self(实例)和cls(类)参数.

```
class SomeClass:
    debug=False
    def __init__(self,x):
        self.x=x
        @staticmethod
    def yowv1():
        print('SomeClass.yow')
```

静态方法的核心价值是命名空间组织和逻辑分组.适用于无状态的辅助函数,与类职责相关但不依赖实例或类的状态.使用静态方法可以提升代码可读性以及模块的内聚性.如果函数不需要考虑实例或类的状态,那么可以定义为静态方法;如果函数未来需要访问类的状态或者支持类继承,那么需要定义为类方法.

综上,类变量通常用于保存全局参数,这个参数在所有实例之间共享.子类继承基类后可以重写相关类变量从而改变类方法行为.类方法最常用是实现备用构造函数.我们常见的类方法名需要有from一词,例如

```
d=dict.fromkeys(['a','b','c'],0)
print(d)
```

4 Class and Encapsulation

类的主要作用是封装数据和对象的内部实现,而同时类也会提供对外的公共接口,用来操纵对象.因此在Python中区分对象内部实现和对外接口十分重要.但是和C/C++通过语法规定强制执行私有/公有,Python的私有封装是通过对属性的命名来约定俗成的指示属性的预期用途.因为Python默认程序员在编程中有义务遵守私有和公有协议,故而并没有做很强制的约束.

用_引导的属性名表示这个属性是私有变量,但是虽然名义上是私有变量,我们仍然可以调用并修改他.

```
class Base:
    def __init__(self,name):
        self._name=name
b=Base('Guido')
print(b._name)
b._name='Dave'
print(b._name)
```

在继承中,子类仍然可以访问父类的私有属性.实际上,_只不过是一个命名约定,表示他约定这个属性只在内部可用,但是不会阻止外部访问.为了避免出现私有属性被外界随意调用,我们可以使用__来引导变量.他与单下划线引导的变量不同的是,Python会使用名称修饰,因此我们不能通过__name的方式直接调用他,而是需要使用_cls__name来调用变量.

```
class Base:
    def __init__(self,name):
        self.__name=name
b=Base('Guido')
print(b._Base__name) # 'Guido'
b._name='Dave'
print(b._name) # 'Dave'
```

上面我们调用_name变量能够返回一个变量,其实是因为在上面的赋值语句中,在实例中创建了一个_name变量.所以就算用子类继承父类,也不能利用__name来调用修改变量,如果一定要修改也只能使用_Parent__name的形式.

对于某些需要考虑变量类型的情况下,我们需要使用引入访问器方法,在私有属性上加上get/set函数,但是不幸的是会破坏已有的代码.

```
class Stock:
    def __init__(self, name, shares, price):
        self.name = name
        self.set_shares(shares)
        self.price = price
    def get_shares(self):
        return self._shares
    def set_shares(self, value):
        if not isinstance(value, int):
            raise TypeError('Expected an int')
        self._shares = value
```

我们希望可以加上类型判断后,并不会影响代码结构,因此引入了property装饰器.

```
class Stock:
    def __init__(self, name, shares, price):
        self.name = name
        self.shares=shares
        self.price = price
        @property
    def shares(self):
        return self._shares
    @shares.setter
    def shares(self,value):
        if isinstance(value,int):
            self._shares=value
        else:
            raise TypeError('Expected int')
```

我们普通的访问属性会自动触发调用方法,例如我们访问shares属性,则会自动访问property装饰的shares方法;通过对shares的赋值,则会自动访问shares.setter装饰的shares初始化方法.所以我们可以知道property装饰器,其实可以把函数也给装饰成普通变量属性的形式,也就是

```
class Stock:
    @property
    def cost(self):
        return self.price*self.shares
```

如果我们正在创建对象并且希望对象的各个属性具有一致的接口.

对于__slots__属性的使用,应该保持谨慎态度,虽然他能够减少内存消耗,但是他在继承上可能会有十分复杂的情况,如果一定要使用,建议用在简单的数据结构类中.

5 Inheritance

继承可以认为是为了扩展现有的对象的方式.我们一般称新类为派生类或子类;父类为基类或超类,父类一般是通过子类后面的括号指定.

```
class Parents:
    pass
class Child(Parents):
    pass
```

继承一般有两个作用,一是向已有代码中添加新的方法,二则是重写父类方法.如果在子类中重写父类方法时需要使用父类方法,那么我们需要利用super()来调用子类的父类.使用子类继承的时候,我们必须先对父类进行初始化.类的继承其实建立了一个类型关系,我们可以通过isinstance来判断相应的关系,我们可以认为继承定义的子类对象其实是父类对象的一个特殊版本.其实Python提供了object类,可以认为Python的任意对象都是object的子类.不止于单继承,我们可以提供多个父类,从而实现多重继承的子类,其会继承多个父类的特性,但是多重继承由于MRO的存在,需要谨慎使用.综上,继承可用做代码的自定义/可扩展特性,向现有代码添加内容,执行自定义处理.

6 Special Method

类通过修改或自定义特殊方法的方式,来定义对象的几乎全部行为.对象有两种字符串模式:str函数会返回适合打印的字符串文本;repr函数会返回具备编程开发信息的字符串文本.repr函数返回的是一个字符串,其可以通过eval函数重新创建一个相应的地政对象.若不存在,这样的字符串则以尽可能易读的形式表示字符串.

实例的创建实际上是分成两步:

1. 先创建一个未初始化的类实例对象

   ```
   d=Date.__new__(Date)
   ```

2. 再调用初始化函数对类实例对象做初始化操作

   ```
   d.__init__(2026,2,3)
   ```

因此我们可以通过调用类的__new__方法来自定义一些跳过初始化的创建函数,代码如下所示

```
import time
class Date:
    @classmethod
    def today(cls):
        tm=time.localtime()
        self=cls.__new__(cls)
        self.year=tm.tm_year
        self.month=tm.tm_mon
        self.day=tm.tm_mday
        return self
```

我们首先需要表明的是,在程序中随意修改类的__new__方法并不被提倡,但我们会在后续中提及某些场景下修改__new__方法的情况.在此我们先介绍一下,__new__方法在类中是以静态方法的形式出现的.这是因为new方法先于实例创建而存在,因此他并不能是实例方法,而new方法不被定义为类方法的原因是,如果是类方法,那么Python就会自动将调用他的类作为第一个参数绑定进入,这可能会影响代码的灵活性,而staticmethod则可以显式的传入一个类对象参数,方便开发者自由决定.

我们继续更详细的介绍new方法的相关用法.

```
class Dog():
    def __new__(cls, *args, **kwargs):
        print("run the new of dog")
        return super().__new__(cls)
    def __init__(self):
        print("run the init of dog")
        print(self)
        print(self.__class__)
a = Dog()
```

上述代码的运行流程是,先查找Dog中的new方法,如果没有找到那就会进一步查找父类的new方法,一直查找到object的new方法.上面的代码中我们查找到Dog中的new方法,其中他会调用super().__new__(cls),也就是调用父类的new方法来创建cls类.这里我们需要注意的是,父类的new方法是可以创建子类实例的,这其实可以认为子类是父类的一种特例.调用父类的new方法中我们可以发现他其实是显式调用了cls类本身,而在init方法中则是显式调用了self实例对象本身.因此new和init还有一个更为重要的区别是,new方法必须要返回一个类对象实例,而init方法则可以没有返回值.

__new__(cls[,…])是对象实例化的时候,所调用的第一个方法,他会返回一个未初始化的类对象实例.而init方法则是用于初始化实例,因此在调用init方法之前,必须要调用new方法.new方法的语法结构如下所示,

```
__new__(cls[,*args,**kwargs])
```

这表明__new__方法的参数至少要有一个类参数,代表实例化的类.这个参数在实例化由Python解释器提供或用户自行显式提供,而后面的参数则会直接传递给init函数.new方法对当前类做实例化,并将实例返回,传给init函数的self,但new方法不代表会自动调用init,只有返回当前类的实例才会调用当前类的初始化.

```
class A(object):
    def __init__(self, *args, **kwargs):
        print("run the init of A")
    def __new__(cls, *args, **kwargs):
        print("run thr new of A")
        return object.__new__(B, *args, **kwargs)
class B(object):
    def __init__(self):
        print("run the init of B")
    def __new__(cls, *args, **kwargs):
        print("run the new of B")
        return object.__new__(cls)
a = A()
print(type(a))
b = B()
print(type(b))
```

这个代码运行的结果是a不会调用类A的初始化,因为a实际上是B类实例,并不会调用类A的初始化函数.其实不仅如此,哪怕创建的实例是父类的实例,也是不会调用初始化函数的,如下所示

```
class Parent:
    def __init__(self):
        print("Parent.__init__ called")
    def __new__(cls, *args, **kwargs):
        print(f"Parent.__new__ called for {cls}")
        return super().__new__(cls)
class Child(Parent):
    def __init__(self):
        print("Child.__init__ called")
    def __new__(cls, *args, **kwargs):
        print("Child.__new__: returning Parent instance!")
        return super().__new__(Child, *args, **kwargs)
c = Child()
print("type(c):", type(c))
```

这里的结果其实也是子类不会自动调用初始化方法,因为他生成的是父类实例,而非子类.

我们一般不会修改new方法,但是如果我们希望继承一些不可变的数据类时(如int,str,tuple),修改new方法可以提供一个自定义这些类的实例化过程的途径.我们罗列一些需要修改new方法的场景以及相应实现.

1. 实现单例模式

```
class Single:
    __instance=None
    @staticmethod
    def __new__(cls,name,age):
        if not cls.__instance:
            cls.__instance=super().__new__(cls)
        return cls.__instance
    def __init__(self,name,age):
        self.name=name
        self.age=age
a=Single('James',41)
b=Single('Hinton',36)
print(id(a)==id(b)) # True
```

虽然他只有一个实例,但是我们发现其实他是经过了两次初始化,为了避免初始化的覆盖问题,我们可以进一步加上一个flag来标注只允许一次初始化.

```
class Single:
    __instance=None
    __first_init=False
    @staticmethod
    def __new__(cls,name,age):
        if not cls.__instance:
            cls.__instance=super().__new__(cls)
        return cls.__instance
    def __init__(self,name,age):
        if not self.__first_init:
            self.name=name
            self.age=age
            self.__first_init=True
a=Single('James',41)
b=Single('Hinton',36)
print(id(a)==id(b)) # True
```

这里面我们其实可以发现一个十分有趣的现象,就是我们直接用了self.__instance来调用相应的属性,而我们在前面提到过这样的属性名会做一个名称修饰,从而导致真实的变量名应该是_class__instance.这是因为这个过程其实是在编译过程中完成的名称修饰,而这些调用本身都是在类定义内部,所以在做编译的时候会整体上的做名称修饰,故而可以直接形式上的调用.

1. 不可变类型子类化(如int,str,tuple)

```
class PositiveInt(int):
    def __new__(cls,value):
        if value<0:
            value=0
        return super().__new__(cls,value)
class UpperStr(str):
    def __new__(cls, content):
        return super().__new__(cls, content.upper())
x=PositiveInt(-5)
print(x)
s = UpperStr("hello")
print(s)  # "HELLO"
```

2. 自定义实例创建逻辑(如对象池,类型转换等)

```
class Shape:
    def __new__(cls,shape_type,*args):
        if shape_type=='circle':
            return super().__new__(Circle)
        elif shape_type=='square':
            return super().__new__(Square)
        return super().__new__(cls)
class Circle(Shape): pass
class Square(Shape): pass
obj = Shape("circle")
print(type(obj))
```

3. 控制实例数量(对象池)

```
class LimitedInstances:
    _pool = []
    _max = 3
    def __new__(cls):
        if len(cls._pool) < cls._max:
            instance = super().__new__(cls)
            cls._pool.append(instance)
            return instance
         return cls._pool[len(cls._pool) % cls._max]
 a=LimitedInstances()
 b=LimitedInstances()
 print(type(a))
```

Python中除了new方法的创建类实例以外,也有__del__方法,其是类对象的一个析构函数.但一般来说他不会调用,当且仅当类对象的引用数为0,也就是程序中用不到这样的类对象的时候才会触发.因此我们需要把他和del操作符区分开,

```
class Test:
    def __del__(self):
        print('删除引用')
t=Test() # refcount=1
c=t # refcount=2
del t # refcount=1
del c # refcount=0, 调用__del__函数
```

我们用一些代码简要介绍一下del方法的用法,因为我们修改del函数的情况非常之少,所以我们会在后面简要提及.

```
class Person(object):
    def __init__(self,name):
        self.name = name
    def __del__(self):
        print("实例对象:%s"%self.name,id(self))
        print("python解释器开始回收%s对象了" % self.name)
        print("类对象",id(Person))
zhangsan  = Person("张三")
print("实例对象张三:",id(zhangsan))
print("------------")
lisi  = Person("李四")
print("实例对象李四:",id(lisi))
```

这里我们虽然没有对实例进行删除操作,但是由于在程序结束的时候会自动释放所有程序的占用内存,故而会在程序最后调用del方法.

```
import time
class Animal(object):
    def __init__(self, name):
        print('__init__方法被调用')
        self.__name = name
    def __del__(self):
        print("__del__方法被调用")
        print("%s对象马上被干掉了..."%self.__name)
dog = Animal("Dogs")
del dog
cat = Animal("Cats")
cat2 = cat
cat3 = cat
print("---马上 删除cat对象")
```

```
del cat
print("---马上 删除cat2对象")
del cat2
print("---马上 删除cat3对象")
del cat3
print("程序2秒钟后结束")
time.sleep(2)
```

上面的程序中的dog对象因为他只有一个引用数,故而在删除自身后会直接调用del方法,但是由于cat对象做了两次赋值,因此其引用数变成了3,我们需要逐步删除已有的引用,最后才会调用del方法.

```
import sys
class Test:
    pass
t=Test()
print(sys.getrefcount(t))
c=t
print(sys.getrefcount(t))
del c
print(sys.getrefcount(t))
del t
```

Python的sys模块中提供了一个可以读取对象引用数的函数,但值得注意的是,他的引用数读取实际上是比实际的引用数大1的,这是因为测试这个对象引用数的函数本身其实就是对象的一个引用.del方法的危险在于他的调用时机不确定,执行环境不完整,异常不可控,用它去做必须释放的资源在Python中是一个设计错误.因为del依赖的是垃圾回收机制,而不是作用域结束,如果出现了引用环,如

```
a = BadExample()
a.self = a    # 引用环
```

这样就算我们del a,他的引用数还是1,不会调用del方法.

为了避免出现上面的这个情况,Python引入了弱引用的概念.弱引用表示引用某个对象但是不会增加相应的引用计数.Python提供了weakref模块来实现相应的功能,他可以用于处理复杂的对象关系和内存管理问题的内存泄露问题.但由于他的使用可能十分复杂,因此我们并不推荐经常使用这个模块.

```python
class Node:
    def __init__(self,value):
        self.value=value
        self.prev=None
        self.next=None
a=Node(1)
b=Node(2)
a.next=b
b.prev=a
del a
del b
```

这里我们即使删除了a,b,内存中仍然还是会保留对象a和b的内存占用,这是因为他们存在一个互相引用的现象.Python的变量和C/C++不同,Python的变量并没有绑定内存,而只是一个引用副本,故而释放引用副本并不会影响内存,所以即使我们释放了a的引用,其实a引用的内存仍然存在.弱引用的作用就是打破这个引用循环使部分引用不增加引用计数,让对象能在无强引用时立即被回收.我们在此介绍一些典型场景和代码实例:

1. 观察者模式是一种行为型设计模式,它定义了一种一对多的依赖关系,当一个对象的状态发生改变时,其所有依赖者都会收到通知并自动更新.

```python
import weakref
class Subject:
    def __init__(self):
        self._observers=[] # 存储弱存储
    def attach(self,observer):
        self._observers.append(weakref.ref(observer))
    def notify(self):
        # 清理已死亡的对象
        alive=[]
        for ref in self._observers:
            obs=ref()
            if obs is not None:
                obs.update()
                alive.append(ref)
                self.observers=alive
class Observer:
    def __init__(self,name):
        self.name=name
    def update(self):
        print(f"{self.name} received update")
subject = Subject()
obs1 = Observer("A")
obs2 = Observer("B")
```

```
subject.attach(obs1)
subject.attach(obs2)
subject.notify()  # A, B 收到通知
del obs1  # obs1 被回收
subject.notify()  # 仅 B 收到通知, obs1 自动清理
```

这里使用弱引用的优势在于Subject不会阻止Observer被回收,从而避免观察者泄露.

1. 缓存(Cache)避免缓存本身成为内存泄露源.

```
import weakref
class WeakCache:
    def __init__(self):
        # key → weakref to value
        # 当 value 无其他强引用时自动消失
        self._cache=weakref.WeakValueDictionary()
    def get(self,key):
        return self._cache.get(key)
    def set(self,key,value):
        self._cache[key]=value
class A:
    pass
cache=WeakCache()
obj=A()
cache.set("key1",obj)
print(cache.get("key1"))
del obj
print(cache.get("key1"))
```

他的优势是缓存不会延长对象生命周期,适合"透明缓存"场景.其中的weakref.WeakValueDictionary,表示字典的值为弱引用.

1. 树/图结构中的父-子反向引用

```
import weakref
class TreeNode:
    def __init__(self, value, parent=None):
        self.value = value
        self.parent = weakref.ref(parent) if parent else None  # 弱引
用
        self.children = []
    def add_child(self, child):
        child.parent = weakref.ref(self)  # 子→父为弱引用
        self.children.append(child)
# 使用
```

```
root = TreeNode("root")
child = TreeNode("child", parent=root)
root.add_child(child)
del root  # root 可被回收（child.parent 是弱引用，不阻止回收）
# child 仍存活（有局部变量引用）
```

其优势为子节点持有父节点的弱引用,避免父子互相强引用形成循环.

上面我们提到了del方法存在一些缺点,所以对于某些必须被释放的资源,我们选择使用with语句来取代del方法.with语句的结构如下所示:

```
with obj as val: # val=obj.__enter__()
    statements
    statements
    # obj.__exit__(ty,val,tb)
```

这里我们可以自定义修改obj的entry/exit方法.给出如下的代码示例,

```
class Manager:
    def __enter__(self):
        print('Entering')
        return self
    def __exit__(self,ty,val,tb):
        print('Leaving')
        if ty:
            print('An exception occurred')
m=Manager()
with m:
    print("Hello World!")
```

注意,ty,val,tb这三个参数包含尚未处理的异常信息(如果有的话).

7 Abstract Base Class

面向对象编程的重要特点就是在于代码复用和代码可拓展.类通常可以被认为是一种设计规范或编程接口.

```
class IStream:
    def read(self, maxbytes=None):
        raise NotImplementedError()
    def write(self, data):
        raise NotImplementedError()
```

此类一般不会直接实例化,而是作为其他对象的基类.因此我们可以将接口定义为抽象基类(Abstract Base Class,ABC),其中抽象基类的一些实现需要从Python的abc模块中继承,如下所示

```python
from abc import ABC,abstractmethod
class IStream(ABC):
    @abstractmethod
    def read(self,maxtypes=None):
        pass
    @abstractmethod
    def write(self,data):
        pass
```

抽象基类是不能够实例化的,因为他其实只是提供了一些方法的占用,需要在继承中重定义方法实现.除非所有抽象方法都被完整的实现,不然是无法实例化类对象.这其实是可以帮助开发者捕获因疏忽带来的编程错误.不仅如此,传统角度来说,我们如果希望判别对象是否为我们期待的接口,需要手动逐个检查验证方法是否存在且是否可调用,

```python
def write_data(data, stream):
    if not (hasattr(stream, 'read') and callable(stream.read) and
            hasattr(stream, 'write') and callable(stream.write)):
        raise TypeError('Expected a Stream with read() and write()
methods')
    ...
#+END_SR
```

这一流程十分复杂且极其容易遗漏,而抽象基类则可以十分方便的简化上述的类型检测,如下所示

```python
#+BEGIN_SRC python
def write_data(data,stream):
    if not isinstance(stream,IStream):
        raise TypeError('Expected a Stream with read() and write()
methods')
```

在抽象基类中,存在一种比较常见的用法就是处理器类.我们有些时候会在代码中实现一些通用算法,然后将一些关键或可拓展的步骤委托给外部提供的处理器对象.这个方式其实就是策略设计模式.策略设计模式表示在程序中定义了一系列算法或策略,并将每个算法封装在独立的类中,使得它们可以互相替换.通过使用策略模式,可以在代码运行时根据需要选择不同的算法封装对象,而不需要修改客户端代码.他的优点是降低了算法类的职责,使各个算法可以独立变化并相互替换.而且使得增加新算法十分容易,降低对原有系统的侵入,从而使得程序可扩展可维护性增强.但是缺点就是程序功能不断丰富的过程中,程序从局部来看变的更复杂了.

我们给出一个简单的处理器类的代码示例

```python
def print_table(records,fields,formatter):
    formatter.headings(fields)
    for r in records:
        rowdata=[getattr(r,fieldname,'undef') for fieldname in fields]
        formatter.row(rowdata)
```

上面的代码中formatter其实就是一个处理器类,他在print_table里面被不断调用,因此使用处理器类的关键在于算法的关键步骤委托给独立的处理器类,从而实现代码的解耦.处理器类和抽象基类一样,都是具有自己的类定义,一般来说他只需要包含需要实现或者自定义的方法.处理器类在Python标准库中极为常见,他的优势是提供了代码的灵活性;处理器类和代码实现解耦;处理器类在其他的代码上下文中可复用.

抽象基类的另一个特例在于构造模板类.模板类实现通用算法,但是将某些特殊化步骤委托给子类实现.如

```
class CSVParser:
    def parse(self, filename):
        records = []
        with open(filename) as f:
            rows = csv.reader(f)
            self.headers = next(rows)
            for row in rows:
                record = self.make_record(row)  # 需子类实现的步骤
                records.append(record)
        return records
    def make_record(self, row):
        raise RuntimeError('Must implement')  # 未实现时抛出错误
```

这种类是无法直接是用的,必须通过子类继承来补充缺失的功能.如下所示

```
class DictCSVParser(CSVParser):
    def make_record(self, row):
        return dict(zip(self.headers, row))
parser = DictCSVParser()
portfolio = parser.parse('portfolio.csv')
```

他的核心思想是用户仅需定义小型子类提供缺失部分,大部分功能由基类提供.但实际上,上面的这个结构通过模板类和子类的结构从代码角度来看其实十分复杂,其实可以直接用函数回调的方式直接替代.

```
def parse_csv(filename, make_record):
    records = []
    with open(filename) as f:
        rows = csv.reader(f)
        headers = next(rows)
        for row in rows:
            record = make_record(headers, row)  # 用户传入回调函数
            records.append(record)
    return records
def make_record(headers, row):  # 用户实现
    ...
```

因为模板模式可能会过于复杂,而函数的回调会更加简洁.

8 Advanced Inheritance

继承是代码复用(定制与扩展)的工具,子类可以对父类已有的方法进行自定义修改与扩展.多重继承会使得子类继承所有父类的特性.Python允许协作式多重继承,子类可以显式安排多个父类协作.子类定义时父类的顺序十分重要,他会直接影响父类在方法解释顺序也就是MRO中的顺序.调用子类属性的时候,会在他的MRO中寻找相应的属性.此外,super()函数会按照MRO依次调用父类的方法,形成协作,这里需要注意的是super()不一定是父类,而是MRO的下一个类.例如,

```
from abc import ABC,abstractmethod
class TableFormatter(ABC):
    @abstractmethod
    def headings(self,headers):
        raise NotImplementedError()
    @abstractmethod
    def row(self,rowdata):
        raise NotImplementedError()
class TextTableFormatter(TableFormatter):
    def headings(self,headers):
        print(' '.join('%10s' % h for h in headers))
        print(('-'*10+' ')*len(headers))
    def row(self,rowdata):
        print(' '.join('%10s' % d for d in rowdata))
class UpperHeadersMixin:
    def headings(self,headers):
        super().headings([h.upper() for h in headers])
class UpperHeadersFormatter(UpperHeadersMixin,TextTableFormatter):
    pass
```

这里的UpperHeadersMixin类的heading方法调用了super方法,他其实并不一定代表UpperHeadersMixin的父类,因为我们会在UpperHeadersFormatter里调用,所以他其实是表示在MRO顺序下往后的某一个类,未必是他自己的父类. Mixin是专门向其他类定义添加额外功能而设计的类.其核心思想是用户实现基础功能后,Mixin可自动补充额外的函数功能.他的主要作用是可以减少代码重复,减少需要编写的代码量.他的典型用途是为基本对象添加可选功能(如线程支持,持久化等),用户通过组合不同部件装配所需对象.

Inside Python Objects

1 Underlying dictionary

字典是命名化值的集合,用户自定义的对象的底层使用字典,如实例数据和类成员.换言之,整个对象系统的底层是基于字典.实例的__dict__字典属性会用于存储实例数据,在创建实例属性并赋值的时候,实际上就是对__dict__字典的元素填充.并且不同实例的__dict__字典属性是私用的,并不会互通共享.换言之,其实就是如果程序中创建了100个实例,那么也会有100个不同的实例字典.与实例的__dict__字典不同的是,类也是存在__dict__字典属性的,其是用于存储类成员.类和对象在逻辑上是具有很强的联系的,也就是实例是类的可操作对象,类提供了实例的数据属性和操作行为.实例的__class__属性返回的是其实例化的类,因此可以从实例反连接到类.

```
class Stock:
    def __init__(self,name,shares,price):
        self.name=name
        self.shares=shares
        self.price=price
    def cost(self):
        return self.price*self.shares
    def sell(self,nshares):
        self.shares-=nshares
print(Stock.__dict__)
s=Stock('GOOG',100,490.10)
print(s.__dict__)
print(s.__class__)
```

实例字典是私有的,而类字典则是所有实例都可以共享.

Python通过.操作符来实现对属性的调用,如下所示

```
x=obj.name # Getting
obj.name=value # Setting
del obj.name # Deleting
```

这些操作其实可以直接反映到底层字典,因此修改和删除实例数据会直接在底层字典上更新.如果我们希望读取实例的某个属性,程序会优先在局部实例字典中查找,如果没有,再查找类字典.这种查找方式会使类成员被所有实例共享.

2 Inheritance

继承在前面已经提到了很多次,这里主要是介绍一下关于多重继承的继承结构和方法调用上.Python允许类从多个父类中继承,其相关的直接父类会以元组的形式存储在每个类的__bases__属性中,但对于父类的父类并不会存储返回.而在类属性中,还有一个类似的属性__base__,他会返回第一个直接父类,一般用于单继承.__bases__属性会提供指向父类的链接,从而可以扩展用于查找属性的搜索过程.因此完整的搜索链其实是先查找局部实例字典,再查找类字典,最后查找父类字典.

在继承层次结构上,属性是通过沿着继承树查找.若只是单继承,则查找线路只有一个直线,程序会在第一个匹配的对象时停止并返回.

```
class A(object): pass
class B(A): pass
class C(A): pass
class D(B): pass
class E(D): pass
print(E.__mro__)
```

这种继承链会在类完成继承的时候预计算,并且存储在类的__mro__属性.多继承的MRO会比较复杂,但是我们在前面已经提到了相关的算法,如感兴趣可以查阅MRO算法.

```
class A(object): pass
class B(object): pass
class C(A,B): pass
class D(B): pass
class E(C,D): pass
print(E.__mro__)
```

在类方法的重写中,如果我们需要调用父类的方法,当然可以直接明确的指出所使用的父类名称,如

```
class Base:
    def spam(self,x):
        print(f"Base.spam({x})")
class A(Base):
    def spam(self,x):
        print(f"A.spam({x})")
        Base.spam(x)
```

这样会很明确,但是如果继承结构的变化会使得程序的修改和维护变得困难.因此为了重写方法,我们一般选择使用super()函数.super()函数会调用给MRO上的下一个匹配的类,而不是直接父类,因此在不同的MRO中,同一个类的super()返回的可能不是同一个类,这里我们需要额外注意.

我们简要介绍一下继承设计的几个原则:第一,需要有兼容的方法参数,重写方法在整个继承结构中必须存在兼容参数..如果方法参数各不相同,那么可以使用关键字参数.代码如下所示

```
class Base:
    def spam(self,x):
        print(f"Base.spam({x})")
class A(Base):
    def spam(self,x):
        print(f"A.spam({x})")
        super().spam(x)
class B(Base):
    def spam(self, x, y):
        print(f"B.spam({x}, {y})")
        super().spam(x)
class C(A, B):
    def spam(self, x, y):
        print(f"C.spam({x}, {y})")
        super().spam(x, y)
try:
    c = C()
    c.spam(1, 2)
except TypeError as e:
    print(f"错误: {e}\n")
```

这里会因为类C调用类A的spam函数,并且传入两个参数,这与类A的spam函数的所需参数不同,因此会报错.故而需要采用关键字参数,如下所示,

```
class BaseFixed:
    def spam(self, x, **kwargs):
        print(f"BaseFixed.spam(x={x})")
        # 消耗掉已处理的参数，传递剩余参数
        kwargs.pop('extra', None)  # 可选：清理特定参数
class AFixed(BaseFixed):
    def spam(self, x, **kwargs):
        print(f"AFixed.spam(x={x})")
        super().spam(x, **kwargs)  # 安全传递所有未使用参数
class BFixed(BaseFixed):
    def spam(self, x, y, **kwargs):  # 可安全添加新参数
        print(f"BFixed.spam(x={x}, y={y})")
```

```
        super().spam(x, **kwargs)
class CFixed(AFixed, BFixed):
    def spam(self, x, y, z=None, **kwargs):
        print(f"CFixed.spam(x={x}, y={y}, z={z})")
        super().spam(x, y=y, z=z, **kwargs)
# 测试: MRO 顺序为 CFixed -> AFixed -> BFixed -> BaseFixed -> object
print("MRO:", [cls.__name__ for cls in CFixed.__mro__])
print()
c_fixed = CFixed()
c_fixed.spam(1, y=2, z=3)
```

关键字参数的存在可以安全的传递一些额外的参数.第二,方法链必须要终止,super()函数不可以永远调用,因为MRO的长度是有限的;第三,在任何地方都可以使用super()函数.

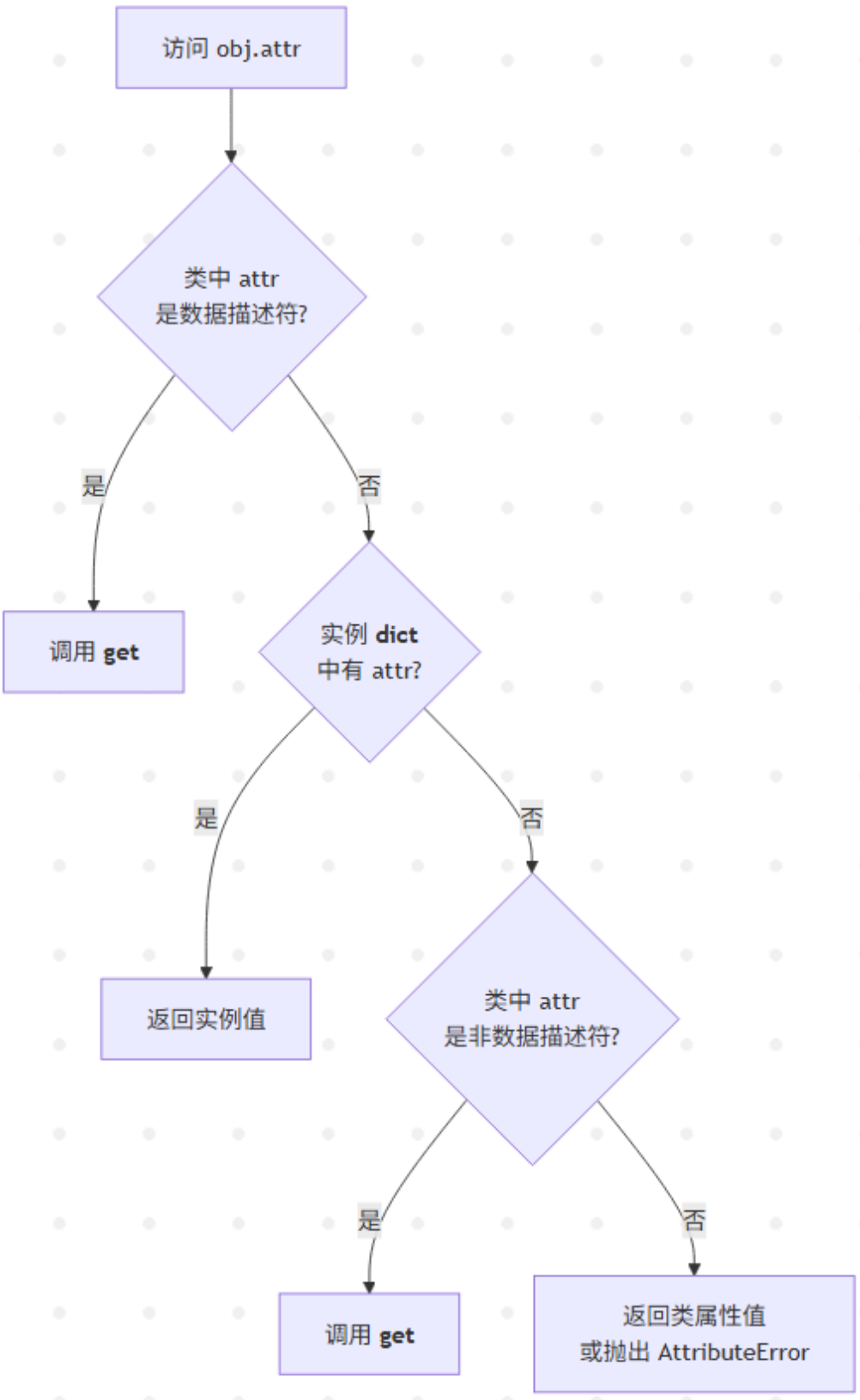
3 Descriptor

描述符是Python最强大但常被忽略的元编程工具,其用于实现property,classmethod等高级特性的底层实现.描述符指的是实现了描述符协议(get,__set__,__delete__中至少一个)的类实例.若类试图访问类中的某个属性,我们会检测其是否具有描述器符特征,如果有,那么他的相关调用就会被描述符覆盖.

```
class Descriptor:
    def __init__(self,name):
        self.name=name
    def __get__(self,instance,owner):
        return instance.__dict__[self.name]
    def __set__(self,instance,value):
        instance.__dict__[self.name]=value
    def __delete__(self,instance):
        pass
class Foo:
    a=Descriptor('a')
```

从上面的代码来看,我们发现描述符的实例其实是类属性,但是他其实操作的是实例字典,也就是他会拦截实例属性的访问行为.

Python属性访问遵循严格优先级,这是理解描述符的关键:



数据描述符指的是实现了__set__或__delete__的描述符类,他会拦截赋值写入;非数据描述符指的则是只实现了__get__方法,他不会拦截对实例字典的访问,相反他会被实例字典覆盖.查找的路线是数据描述符,实例字典,非数据描述符,类字典.给出如下的代码实例

```
class DataDesc:
    def __get__(self, instance, owner):
        print(f'__get__: {instance=}, {owner=}')
        return instance.__dict__.get('attr', 'default')
    def __set__(self, instance, value):
        print(f'__set__: {value=}')
        instance.__dict__['attr'] = value
class A:
    attr = DataDesc()  # 数据描述符（有 __set__）
a = A()
a.attr = 42          # → __set__: value=42（被拦截）
print(a.attr)        # → __get__: instance=<A...>, owner=<class 'A'> →
42
print(a.__dict__)    # → {'attr': 42}
```

上面的a.attr试图访问的是实例字典的属性,但是被数据描述符拦截,从而调用数据描述符的方法.

```
class NonDataDesc:
    def __get__(self, instance, owner):
        print('__get__ called')
        return 'from descriptor'
class B:
    attr = NonDataDesc()  # 非数据描述符（无 __set__）
b = B()
print(b.attr)
b.attr = 'instance value'  # 直接写入实例字典，不触发 __set__
print(b.attr)              # → 'instance value'（实例字典优先）
print(B.attr)              # → __get__ called → 'from descriptor'
```

这里我们先调用b.attr时,发现他会调用非数据描述符的get方法,如果我们对b.attr赋值以后,那么他的相关调用就会被实例字典覆盖,除非我们用类属性的方式调用,因为非数据描述符本质上是一个类属性.

从上面的代码,我们可以看出描述符其实是包含了实例,类和数值的信息.在程序编写中我们需要注意的是,self是描述符本身,instance则表示是操作的实例对象.虽然他的调用从形式上来看和实例字典是一样的,但是我们可以把他用如下的方式拆解.

```
q=A.__dict__['attr']
print(q) # <__main__.DataDesc object at 0x00000279B19DA930>
q.__set__(a,43) # __set__: value=43
print(q.__get__(a,a.__class__))
# __get__: instance=<__main__.A object at 0x000001FCD0FDA9F0>, owner=
<class '__main__.A'>
# 43
```

我们通过代码实现可以知道,描述符是可以存储和检索数据的,他会直接操作实例底层字典,来实现对数据的读写操作.这也就是我们前面提到的,描述符是一个类属性,但是他的操作对象则是实例对象的底层字典.因此许多重要的类行为都是通过描述符实现的,例如实例方法,静态方法,类方法以及property装饰器,我们在后续会给出一些自定义的实现.我们前面提到过函数的调用是分成两个步骤的,先通过.操作符查找到绑定方法,再用()操作符去调用相关方法.就此我们用描述符的角度去解释一下

```
import stock
s=stock.Stock('GOOG',100,490.10)
# class attribute lookup
value=stock.Stock.__dict__['cost']
print(value)
#descriptor check
print(hasattr(value,'__get__')) # true
print(hasattr(value,'__set__')) # false
print(hasattr(value,'__delete__')) # false
#invocation
result=value.__get__(s,stock.Stock)
print(result)
print(result())
```

故而可知value其实就是一个非数据描述符,对其进行赋值后,会返回一个绑定方法,但这个其实只是一个返回设定,如果我们自定义了一个返回为字符串,那么就是字符串,只是我们可以通过描述符进一步解释实例方法的调用.因此非数据描述符被激发的前提是在实例字典上没有匹配的属性,他才会被激发.

__slots__和__dict__不同的是他在实例创建之初,就会预先检查类字典中是否有同名属性,如有则报错,若没有,那么就会计算分配一个数组空间.从底层来看,每个slot名都会创建一个描述符用来完成在指定的位置进行简单的读取写入操作.但我们仍然可以自定义一个描述符,但需要注意的是如果我们希望可以实现对slots的描述符,那么类属性和slots必须要分开,不然是无法创建slots数组的,我们给出正确的代码示例.

```
class Descriptor:
    def __init__(self,name):
        self.name='_'+name
    def __get__(self,instance,cls):
        if instance==None:
            return self
        return getattr(instance,self.name)
    def __set__(self,instance,value):
        setattr(instance,self.name,value)
class A:
    __slots__=('_x','_y')
```

```
    x=Descriptor('x')
    y=Descriptor('y')
a=A()
a.x=10
a.y=20
print(a.x)
print(a.y)
```

这里我们不用前面常见的操作底层字典的方式是因为slots并没有像底层字典一样提供存储空间,而是向程序宣告允许的属性名是什么,其相关的内部空间由底层程序控制,因此我们需要调用setattr和getattr等属性访问函数.其次,这里的类A中在类字典中是具有属性x和y的,而对于实例slots的_x和_y则是通过调用类属性的数据描述符x和y操作.同样slots也是支持删除操作的,因为slots的主要目的其实是为了内存优化和属性限制,也就是控制某些属性的存在.

描述符的主要作用其实是描述数据,提供比property装饰器更为精细的控制,可以使用更少的重复代码.

```
import numbers
class Integer:
    def __init__(self,name):
        self.name=name
    def __set__(self,instance,value):
        if not isinstance(value,int):
            raise TypeError('Expected an integer')
        instance.__dict__[self.name]=value
class String:
    def __init__(self,name,maxlen):
        self.name=name
        self.maxlen=maxlen
    def __get__(self,instance,cls):
        return instance.__dict__[self.name]
    def __set__(self,instance,value):
        if not isinstance(value,str):
            raise TypeError('Expected a string')
        if len(value)>self.maxlen:
            raise TypeError('The length must be less than 8')
        instance.__dict__[self.name]=value
class Real:
    def __init__(self,name):
        self.name=name
    def __get__(self,instance,cls):
        return instance.__dict__[self.name]
    def __set__(self,instance,value):
        if not isinstance(value,(int,float)):
```

```
            raise TypeError('Expected an integer or float')
        if not isinstance(value,numbers.Real):
            raise TypeError('The number must be real')
        instance.__dict__[self.name]=value
class Stock:
    name=String('name',maxlen=8)
    shares=Integer('shares')
    price=Real('price')
s=Stock()
s.name='Hello'
print(s.name)
```

这里我们可以发现Integer其实没有设置get方法,这是因为Python内部其实也是有get方法的,如果我们在描述符内不去设置get方法,那么程序就会调用程序自带的get方法,这要求我们所调用的名字和实例字典里的名字需要完全一样.

```
class Integer:
    def __init__(self,name):
        self.name='_'+name
s.shares=1
print(s.shares) # <__main__.Integer object at 0x000002A278692300>
print(s._shares) # 1
```

其实如果我们省略了get方法的话,可以认为就是直接去访问实例字典.get方法可以被两种方式调用,一是通过绑定实例的方式,也就是直接使用实例调用;二则是通过类调用,这样他不会绑定实例.这里我们可以知道上面的描述符对get方法的实现存在一个问题,因为它本质上是一个类方法,但是我们的实现中只考虑实例调用的情况,所以我们需要补充一下get方法,

```
class Descriptor:
    def __get__(self,instance,cls):
        if instance is None:
            # If no instance given, return the descriptor
            # object itself
            return self
        else:
            # Return the instance value
            return instance.__dict__[self.name]
```

我们在前面中定义描述符的时候,实例属性的名称需要显示给出,但一般而言我们为了方便起见都会定义成原来的名字,如果多次出现,会使得代码十分臃肿.因此Python3.6版本以上提供了名称设置器的方法以简化描述符的实例化.

```python
class Descriptor:
    def __init__(self, name=None):
        self.name = name
    def __get__(self, instance, cls):
        if instance is None:
            return self
        return instance.__dict__[self.name]
    def __set_name__(self, cls, name):
        self.name = name
class Spam(object):
    x=Descriptor() # x.__set_name__(Spam,'x')
```

我们可以让类的多个属性分别绑定在同一个描述符,但是不可以让同一个属性绑定在多个描述符上,如果发生了,那么后定义的会覆盖前定义的.

```python
class Integer:
    def __init__(self, name=None):
        self.storage_name = name if name is None else '_' + name
    def __set_name__(self, owner, name):
        # Python 3.6+ 自动捕获属性名（推荐）
        if self.storage_name is None:
            self.storage_name = '_' + name
    def __get__(self, instance, owner):
        if instance is None:
            return self
        return getattr(instance, self.storage_name)
    def __set__(self, instance, value):
        if not isinstance(value, int):
            raise TypeError(f"{self.storage_name} must be int")
        setattr(instance, self.storage_name, value)
class Point:
    x = Integer()  # 实例1：管理 x
    y = Integer()  # 实例2：管理 y
    def __init__(self, x, y):
        self.x = x
        self.y = y
p = Point(10, 20)
print(p.x, p.y)  # 10 20
p.x = 30         # → Integer实例1的__set__
p.y = 40         # → Integer实例2的__set__
print(p.x, p.y)  # 30 40
```

这里x和y是两个独立的Integer实例,并且他们通过__set_name__方法来获得唯一的实例属性名,他们不会互相干扰,各自管理自己的属性.

我们给出一些描述符的简单实现,覆盖静态方法,类方法,property装饰器等场景.

```
class Mystatic:
    def __init__(self,func):
        self.func=func
    def __get__(self,instance,owner):
        # 无论是实例访问还是类访问都返回函数
        return self.func
class Math:
    @Mystatic
    def add(x,y):
        return x+y
print(Math.add(1,2))
m=Math()
print(m.add(2,3))
```

这里的特点就是get方法返回的是self.func,并没有和任何类,实例进行绑定.因此对这个方法而言,其实在实现逻辑上并不属于类和实例,只是在编程逻辑上应该属于类.

```
class Myclass:
    def __init__(self,func):
        self.func=func
    def __get__(self,instance,cls):
        def wrapper(*args,**kwargs):
            return self.func(cls,*args,**kwargs)
        return wrapper
class Date:
    def __init__(self, year, month, day):
        self.year = year
        self.month = month
        self.day = day
    @Myclass
    def today(cls):
        import datetime
        now = datetime.date.today()
        return cls(now.year, now.month, now.day)
d = Date.today()
print(d.year)     # 2026
```

属性访问Date.today触发描述符协议,调用MyClass.__get__(None,Date),此时get方法会返回一个闭包wrapper,他捕获了owner=Date和原始函数self.func.此时调用Date.today,其实就是调用的就是wrapper函数,只是wrapper函数已经给定了第一个参数为类,所以他其实就是一个类方法.

```
class Myproperty:
    def __init__(self,fget=None,fset=None,fdel=None,doc=None):
```

```
        self.fget=fget # gettr 函数
        self.fset=fset # settr 函数
        self.fdel=fdel # deleter 函数
        self.__doc__=doc or (fget.__doc__ if fget else None)
    def __get__(self,instance,owner):
        if instance is None:
            return self
        if self.fget is None:
            raise AttributeError("unreadable attribute")
        return self.fget(instance)  # 调用 getter
    def __set__(self,instance,value):
        if self.fset is None:
            raise AttributeError
        self.fset(instance,value)
    def __delete__(self,instance):
        if self.deleter is None:
            raise AttributeError
        self.fdel(instance)  # 调用 deleter
    def setter(self, fset):
        return type(self)(self.fget, fset, self.fdel, self.__doc__)
    def deleter(self, fdel):
        return type(self)(self.fget, self.fset, fdel, self.__doc__)
class Circle:
    def __init__(self, radius):
        self._radius = radius
    @Myproperty
    def radius(self):
        """圆的半径"""
        return self._radius
    @radius.setter
    def radius(self, value):
        if value < 0:
            raise ValueError("半径不能为负")
        self._radius = value
    @radius.deleter
    def radius(self):
        del self._radius
c = Circle(5)
print(c.radius)    # 5 (触发 __get__ → fget)
c.radius = 10      # 触发 __set__ → fset
del c.radius       # 触发 __delete__ → fdel
```

我们首先用Myproperty装饰器装饰了radius函数,相当于用了如下代码,

```
radius=Myproperty(radius)
```

此时的radius就是Myproperty的一个实例对象.而后的radius.setter和deleter用来填充需要的其他相关代码.

```
class CachedProperty:
    def __init__(self,func):
        self.__name__=func.__name__
    def __get__(self,instance,owner):
        if instance is None:
            return self
        print('Called get function')
        value=self.func(instance)
        setattr(instance,self.__name__,value)
        return value
class MyClass:
    def __init__(self,data):
        self._data=data
    @CachedProperty
    def processed_data(self):
        result=[x**2 for x in self._data]
        return result
obj=MyClass([1,2,3,4,5])
print(obj.processed_data)
# Called get function
# [1, 4, 9, 16, 25]
print(obj.processed_data)
# [1, 4, 9, 16, 25]
```

这里我们发现他调用了两次相同的函数,但实际上非数据描述符只调用了一次,因为调用非数据描述符的时候在实例字典中设置了一个同名属性,从而覆盖非数据描述符的调用.

4 Attribute Access Methods

类可以通过实现特定的特殊方法来拦截对属性的访问操作,如获取(get),设置(set)和删除(delete).其中的调用流程大体如下所示,获取流程:我们调用obj.x,在程序内部会先试用obj.__getattribute__('x'),如果程序没有找到相应的属性,则会调用obj.__getattr__('x')作为后备安全方法.设置流程:我们调用obj.x=val,在程序内部会直接调用obj.__setattr__('x',val).删除流程:我们调用del obj.x,在程序内部会直接调用obj.__delattr__('x').

getattribute方法的语法结构如下所示,

```
__getattribute__(self,name)
```

每次读取属性时都会调用这个方法,无论属性是否存在.他的默认行为是按顺序查找描述符,而后是实例字典,最后则是基类(沿着MRO向上查找).如果上面的查找流程都没有查找的同名属性,那么就会调用__getattr__作为备用函数.

getattr方法的语法结构如下所示,

```
__getattr__(self,name)
```

他的整个调用方式和上面的getattribute方法一致,他只有在上面的getattribute方法查找不到的时候才会被调用.其默认行为是会抛出异常AttributeError.我们可以自定义修改设置,从而适合实现动态属性,默认值返回,代理转发等功能.

我们用如下的代码解释二者的区别,

```
class Demo:
    def __init__(self):
        self.existing = 'I exist'
    def __getattribute__(self, name):
        print(f'__getattribute__ 被调用: "{name}"')
        try:
            return super().__getattribute__(name)
        except AttributeError:
            print(f"标准查找失败,需要触发getattr")
            raise
    def __getattr__(self,name):
        print(f"__getattr__ 被调用（后备）: '{name}'")
        return f"默认值: {name}"
obj=Demo()
print("\n--- 访问存在的属性 ---")
print(obj.existing)
print("\n--- 访问不存在的属性 ---")
print(obj.missing)
```

上面我们可以发现,存在的属性会直接调用getattribute方法,并能够查找直接返回值.而缺失的属性则是getattribute方法同样也会被调用,但是他会返回一个异常,同时自动调用getattr方法.所以这两个方法的核心区别是__getattribute__方法的优先级是最高的,只要读取属性就不需要被调用;__getattr__方法仅在属性缺失的时候作为后备被调用.

setattr方法的语法结构如下,

```
__setattr__(self,name,value)
```

其只在每次对属性赋值的时候才会调用.默认行为是检查是否为数据描述符,如果是,那么就调用内部的set方法,否则,就会在实例字典中存入到相应属性中.delattr方法的语法结构如下,

```
__delattr__(self,name)
```

其只在属性删除的时候才会被调用.默认行为是检查是否为数据描述符,如果是,那么就会调用内部的delete方法,否则,就会从实例字典中删除属性.上述的类方法都可以通过重定义的方式来自定义属性访问方法,以符合更多的应用场景,如创建包装对象,代理或其他的情况.

代理转发场景

```
class Proxy:
    def __init__(self,obj):
        self._obj=obj
    def __getattr__(self,name):
        print('getattr: ',name)
        return getattr(self._obj,name)
class Circle:
    def __init__(self,rad):
        self.radius=rad
    def area(self):
        import math
        return math.pi*self.radius**2
c=Circle(4.0)
print(c.radius)
print(c.area())
p=Proxy(c)
print(p)
print(p.radius)
print(p.area())
```

这里的Proxy持有内部对象_obj,其会用于存储外部的类,从而形成对_obj的代理访问.所有不存在于Proxy类自身的属性访问都会触发__getatttr__,同时将调用请求转发给内部对象_obj.适合于实现日志记录,权限控制,远程调用等包装逻辑.

委托模式场景

```
class A:
    def foo(self):
        print('A.foo')
    def bar(self):
        print('A.bar')
class B:
    def __init__(self):
        self._a=A()
    def bar(self):
        print('B.bar')
        self._a.bar()
```

```
    def __getattr__(self,name):
        return getattr(self._a,name)
b=B()
b.foo()# 输出: A.foo(委托给A)
b.bar()  # 输出: B.bar → A.bar(B自定义+委托)
```

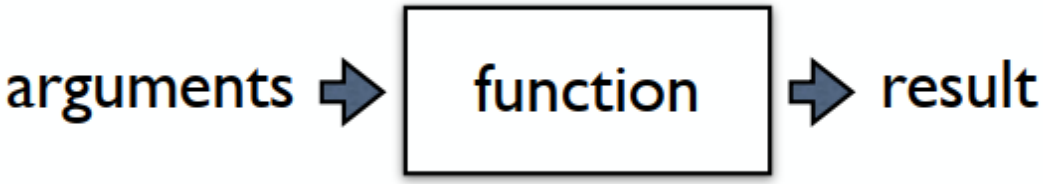
委托可以作为继承的一个替代方案,从而实现更为灵活的组合式设计.需要注意的是,__getattr__不会拦截特殊方法,例如__len__,__getitem__等,如果需要支持,必须要显式实现,

```
class B:
    def __init__(self):
        self._a=A()
    def __getitem__(self,index):
        return self._a[index]
    def __getattr__(self):
        return getattr(self._a),name)
```

Functions

1 Function Arguments

函数是程序的基本构建单元,如模块的顶层函数,类的方法.程序的几乎所有代码应该封装在函数中.Python函数的设计原则,我们希望函数是自包含的(self-contained),仅操作输入的参数.



如果输入相同的参数,则应返回相同的结果,这也就是所谓的函数可预测性.我们在编写程序中需要尽可能避免潜在的副作用,其核心目标是简洁性和可预测性.

函数名和函数参数都是对外的调用接口.调用函数的形式应该直观易用,这是设计良好API的关键.同时我们也可以进一步声明一下函数的命名规范,推荐采用小写字母和下划线的方式来命名函数,而不应采用驼峰命名法,如

```
def read_data(filename):
    pass
```

而一般用于_引导的函数名来指示这个函数是内部或私有函数,如

```
def _internal_func():
    pass
```

在程序运行中,函数可能需要设置默认参数,如

```
def read_data(filename,debug=False):
    pass
```

若默认参数已被设置,则在函数调用的时候,可以选择输入相应参数.就算不输入,函数也会基于默认参数进行函数流程.需要强调的是,默认参数必须在函数参数定义的最后位置.这里我们推荐采用关键字参数来设置可选参数,使用关键字参数传参会使代码可读性提升,我们可以用*来强制使之后的参数调用.同时对于默认参数而言,我们需要尽可能不要用可变类型做默认参数,因为默认参数整个程序只会创建一次,因此程序会有粘性,即存在不可知的修改,

```
def func(a,items=[]):
    items.append(a)
    return items
print(func.__defaults__)
print(func(1))
print(func.__defaults__)
print(func(2))
print(func.__defaults__)
```

默认参数会存放在函数的__defaults__属性中,如果默认参数可变的话,那函数的__defaults__属性中就会不断地记录之前的修改,而我们应该采用不可变值,如None,True,False,numbers或strings.如果默认参数是不可变值,那么默认参数在不断调用中保持原值.

```
def func(a, items=None):
    if items is None:
        items = []
    items.append(a)
    return items
print(func.__defaults__)
print(func(1))
print(func.__defaults__)
print(func(2))
print(func.__defaults__)
```

用不可变的属性可以避免可变类型带来的意外改变.一般来说,可选值用None来表示该参数并未赋值,但需要注意的是,要对是否赋值做错误判断,不然容易报错.函数的设计需要能够接受抽象接口(如可迭代对象),这样会比绑定具体类型(如文件名)更具扩展性.我们也可以采用三个引号包括的字符串作为文档字符串,可以用help函数来查看,是良好的文档实践.我们可以用冒号来引导函数参数的希望类型,用->来引导返回值希望类型.但是这些符号的作用只是辅助代码检查器,IDE和文档生成,Python程序并不会在运行时强制类型检查.

2 Returning Value

函数应该清晰的返回相应值,若需要返回多个值,那么我们需要采用元组的方式进行,调用方则可以用多重赋值的方式解包.

```
def divide(x,y):
    quotient=x//y
    remainder=x%y
    return(quotient,remainder)
print(divide(8,3))
q,r=divide(8,3)
print(q,r)
```

函数的返回值应该是可选的,常见的约定方式是None,这是正常的情况,只是返回的结果不存在;如果发生程序中的错误,则需要抛出异常.我们不应该滥用None来代替异常处理,调用方需要显式检查返回值是否为None,避免出现AttributeErrror异常.

Python提供了允许函数并发执行的方式(通过线程完成),而且多个函数在同一个解释器中并发运行,共享状态.

```
from threading import Thread
import threading, time
def task(name):
    print(f"[{name}] start @ {time.time():.2f}")
    time.sleep(1)          # 模拟 I/O (如网络、磁盘)
    print(f"[{name}] end   @ {time.time():.2f}")
# 顺序执行 (2秒)
start = time.time()
task("A"); task("B")
print(f"Sequential: {time.time() - start:.2f}s")
# 并发执行 (≈1秒)
start = time.time()
t1 = threading.Thread(target=task, args=("A",))
t2 = threading.Thread(target=task, args=("B",))
t1.start(); t2.start()
t1.join(); t2.join()
print(f"Concurrent: {time.time() - start:.2f}s")
```

顺序执行的流程就是简单的完成一个函数之后再调用另一个函数,因此需要执行2s才能够完成执行流程.其关键在于只有一个线程(主线程),任务必须等前一个完成之后才能开始,资源利用率很低.并发执行的流程中,我们调用了time.sleep(1)用于模拟I/O操作,线程在此会释放GIL,允许其他线程运行.我们简要的解释一下流程,线程A,B同时被激发,线程A先抢到GIL,完成输出语句,遇到time.sleep(1)阻塞,操作系统将GIL让给线程B完成,同时完成输出语句后,遇到time.sleep(1)

阻塞,等二者阻塞完成,再重新分配GIL完成输出,只是这个分配过程时间很短,可以几乎认为是同步的.

Python的threading模块支持多线程并发,其中的Thread是相应的线程类,其调用的语法结构,

```
from threading import Thread
Thread(target=func,args=(name,)
```

target用来指定线程执行的函数,不需要加括号,只是传递函数本身,而不是做函数调用.args用来指定传递给目标函数的参数,其必须是元组,如果是单元素就需要加逗号,此时会返回一个未启动的线程对象,此时线程处于新建状态,并没有占用系统资源.start函数表示真正启动线程,其状态会从新建改为就绪而后运行,依靠操作系统来调度.其关键特性在于非阻塞调用且每个线程对象只能start一次.阻塞调用指的是调用后当前线程暂停一直到操作完成完成才继续执行;非阻塞调用指的是调用后立即返回,当前线程可以继续做其他的事情.

```
print("开始")
t1 = Thread(target=task, args=("A",))
t1.start()  # ← 非阻塞：立即返回！
print("继续执行其他代码")  # 这行会立刻执行，不等 task 完成
```

不用run函数的原因是如果调用run则会在主线程中直接调用函数,不会创建新线程,从而失去并发意义.为了让主线程等待线程完成,使用join函数阻塞主线程,直到目标线程执行完毕,其相应的语法结构如下

```
join(timeout=5)
```

其中的可选参数timeout表示最多等timeout秒,超时后主线程继续,但子线程依然在后台运行.

我们继续介绍一些关于并发执行的易错点.下面的代码将展示共享状态和竞态条件,

```
counter = 0
num_threads = 100
def unsafe_increment():
    global counter
    for _ in range(10000):          # 增加次数
        current = counter           # 1. 读
        time.sleep(0)               # 2. 强制线程切换点（关键！）
        current += 1                # 3. 改
        counter = current           # 4. 写
threads = [threading.Thread(target=unsafe_increment) for _ in
range(num_threads)]
for t in threads: t.start()
for t in threads: t.join()
expected = num_threads * 10000
print(f"Expected: {expected}") # 1000000
```

```
print(f"Actual  : {counter}") # 10563
print(f"Lost    : {expected - counter}
```

其根本原因是counter+=1被拆成了四步非原子操作,线程切换发生在中间的步骤,从而导致多个线程覆盖彼此的修改.我们以两个线程A,B为例,假设初始counter=0,A先读取counter为0存储在current,遇到暂停让出CPU;B同时也读取了一个counter为0存储在current,遇到暂停让出CPU;A恢复执行后current+=1,current变为1,同时counter=current=1;B重新恢复执行流程一样,所以两次自增却会丢失一次自加.这里我们用time.sleep(0)来强制当前线程主动让出CPU,从而提高线程在读写改过程中中间被切换的概率,从而使得竞态条件百分百可复现.原子操作指的是不可分割的操作:要么完全执行,要么完全不执行,中间不会被线程切换打断;非原子操作指的是可分割的操作:由多步组成,执行中途可能被其他线程打断.

为了避免上面的竞态条件,我们可以引入锁的概念

```
counter = 0
lock = threading.Lock()
num_threads = 100
def safe_increment():
    global counter
    for _ in range(10000):
        with lock:       # 串行化临界区
            counter += 1
threads = [threading.Thread(target=safe_increment) for _ in
range(num_threads)]
for t in threads: t.start()
for t in threads: t.join()
print(f"With lock: {counter}")  # 稳定输出 100000
```

锁会保护共享状态,保证临界区(修改共享数据的代码)同一时刻只有一个线程在执行.with lock语句则是会自动加锁和释放锁,从而避免死锁.wit lock的行为是尝试获取锁,如果成功就进入临界区,如果失败则会阻塞等待;在临界区内的代码,就会独占执行,其他的线程无法进入;退出with块,自动释放锁,等待队列中一个线程被唤醒获取锁.所以这样使得counter+=1在逻辑上完成原子化不可分割.整个程序的执行方式变为临界区串行和非临界区并发的方式,锁包裹的语句导致形成逻辑原子,不可拆分,仅在锁释放后才可以转换;他的运行性能较差,但是正确性没问题.加锁的性能消耗主要在临界区的串行化和频繁的锁竞争开销,因此锁适用于临界区较大且竞争不激烈的场景,如果我们以高频小操作为主,应该优先考虑无锁算法.

上面提到的threading模块支持的多线程并发,能够使I/O密集型场景(如网络请求,文件读写)完成并发执行,但是对于CPU密集型任务是无法实现的.因为会受到GIL(全局解释器锁)的限制,线程会在I/O时释放GIL,可以分配给其他线程,并发执行;而如果是CPU任务,则线程会持续占用GIL,无法并发执行.GIL指的是CPython中同一时刻中仅一个线程执行字节码.示例代码如下:

```
def cpu_bound(n):
```

```
    while n > 0: n -= 1  # 纯计算，不释放 GIL
# 单线程
start = time.time()
cpu_bound(50_000_000)
print(f"Single thread: {time.time() - start:.2f}s")
# 双线程（受 GIL 限制）
start = time.time()
t1 = threading.Thread(target=cpu_bound, args=(25_000_000,))
t2 = threading.Thread(target=cpu_bound, args=(25_000_000,))
t1.start(); t2.start(); t1.join(); t2.join()
print(f"Two threads  : {time.time() - start:.2f}s")
def io_task():
    time.sleep(1)  # 模拟 I/O（网络/磁盘），此时 GIL 会释放
# 单线程：4 秒
start = time.time()
for _ in range(4): io_task()
print(f"Single: {time.time()-start:.2f}s")
# 双线程：≈1 秒
start = time.time()
t1 = threading.Thread(target=io_task)
t2 = threading.Thread(target=io_task)
t1.start(); t2.start(); t1.join(); t2.join()
print(f"Two threads: {time.time()-start:.2f}s")
```

Python中提供了一个multiprocessing模块,其创建的并非线程而是多进程,从而可以对CPU密集型任务实现并行执行,因为每个进程都独立有GIL.示例代码如下:

```
import time
from multiprocessing import Process, Value
def cpu_bound(n, counter):
    """纯 CPU 密集型任务"""
    while n > 0:
        n -= 1
        counter.value += 1
if __name__ == '__main__':
    # 单进程测试
    counter1 = Value('i', 0)  # 共享整数
    start = time.time()
    cpu_bound(500000, counter1)
    print(f"Single process: {time.time() - start:.2f}s, count=
{counter1.value}")
    # 双进程测试（真正并行）
    counter2 = Value('i', 0)
    counter3 = Value('i', 0)
```

```
    start = time.time()
    p1 = Process(target=cpu_bound, args=(250000, counter2))
    p2 = Process(target=cpu_bound, args=(250000, counter3))
    p1.start()
    p2.start()
    p1.join()
    p2.join()
    total = counter2.value + counter3.value
    print(f"Two processes : {time.time() - start:.2f}s, count={total}")
```

单进程测试中纯CPU计算,没有涉及I/O等待,所以无线程切换,单核百分百占用.Value函数的作用是创建共享内存,虽然单进程的时候这个操作是冗余的,因为他不需要出现进程间的共享内存,所以其实也可以采用普通变量.其调用的语法结构:

```
Value('i',0)
```

Value的第一个参数表示创建的内存变量类型,i表示整数,.d表示double类型,c表示char类型等.第二个参数则是设置的共享内存变量的初始值,访问/修改参数则用.value属性进行修改,

```
counter.value+=1
```

本例中双进程使用两个独立Value,而非共享同一个.可以避免进程间的锁竞争,最大化并行效率.Process的语法结构和thread差不太多,不再赘述,但是他得到的是一个进程,而不是线程.程序执行流程是操作系统会创建两个独立进程,每个进程都有独立Python解释器和独立GIL,进程同时运行在不同的CPU核心上,无GIL争抢,计算真正的并行;join使得主进程阻塞等待子进程结束,即等到两个子进程计算完成.最后从两个共享内存中读取结果.这里用了main模块,是因为windows系统下没有fork()系统调用,需要重新导入主模块创建子进程,不然容易无限递归创建进程导致运行时间过长,Unix下可以省略.

在此我们解释一下进程和线程的区别.进程指的是操作系统分配资源的基本单位,拥有独立内存空间;线程则是CPU调度的基本单位,属于进程,共享进程内存.进程的内存空间是独立的不共享的,线程的空间则是共享所属进程的内存;进程的创建开销更大,因为它需要分配独立的解释器和GIL,复制相应内存,而线程的创建只需要分配调用栈,因此创建更快;进程的隔离性更好,因为单独的进程崩溃不会影响其他的进程,而线程的崩溃会导致整个所属进程挂掉,从而影响其余的进程;进程的安全性很高,因为他们的所属内存是天然隔离的,无需用锁,线程的安全性则并不高,因为共享内存需要手动同步,而且需要加锁对临界区加以保护;进程因为会分配独立的GIL,故而可以处理CPU的并行任务,而线程会因为需要竞争同一进程的GIL,导致CPU并发任务受限,更适合于I/O密集任务.

我们上面介绍了一些进程线程的知识,知道了一些并行操作的方式,但实际上程序运行中很容易出现异步操作的情况,也就是函数A需要调用函数B的结果,但是函数B此时还没有计算到这个结果,需要让函数等待.这里Python提供了Future类来保证异步操作的可行性.

```python
from concurrent.futures import Future
fut=Future()
# 等待结果
value=future.result()
# 设置结果(通常由另一个线程调用)
fut.set_result(value)
```

此时需要协调机制确保结果就绪后再调用.result().future是并发编程中的协调原语,代表异步操作的最终结果.result函数会阻塞当前线程,一直到结果被设置.常用于线程池,异步I/O等场景,是为了构建更高层的并发抽象的基础.

```python
import time, threading
from concurrent.futures import Future
def func(x, y, fut):
    time.sleep(20)
    fut.set_result(x + y)
def caller():
    fut = Future()
    threading.Thread(target=func, args=(2, 3, fut)).start()
    result = fut.result()  # 阻塞20秒
    print('Got:', result)
```

上例表示两个函数的并发执行,且这个模式会广泛应用于线程,异步,多进程等场景.主线程在start启动子线程后遇到result阻塞等待,子线程执行耗时操作后通过.set_result通知主线程,主线程恢复执行并获取结果.

3 Functional Programming

函数式编程风格是以函数为核心构建单元,不存在函数副作用,没有不可变数据,存在高阶函数(允许接受或返回函数的函数).函数式编程强调采用纯函数:相同的输入始终产生相同的输出,不修改外部状态;不可变性:避免修改原始数据,而是生成新数据;组合性:通过小函数组合构建复杂逻辑.Python虽然不是纯函数式编程语言,但是Python的高级特性是允许函数作为参数输入和输出,所以他其实也是支持函数式编程风格的.高阶函数的核心特征是函数可以接受其他函数作为参数,函数可以返回函数作为结果.我们以如下的两个相似函数为例,

```
def sum_squares(nums):
    total=0
    for n in nums:
        total+=n*n
    return total
def sum_cubes(nums):
    total=0
    for n in nums:
        total+=n**3
    return total
```

上例中其实代码的基本结构是一样的,只有累加的函数不同.我们可以将相同的部分抽象成一个公共的函数体,通过输入函数来提供函数的差异部分,这样可以实现代码复用,提高代码可读性,回调函数是高阶函数的典型应用.

```
def sum_func(nums,*,func=None):
    if func is None:
        raise Exception
    total=0
    for n in nums:
        total+=func(n)
    return total
```

单句表达式可以用lambda定义出函数,也就是lambda函数,可以创建匿名函数,即时创建即时定义.我们需要注意的是lambda函数仅能包含单一表达式,不支持控制流,异常等.lambda函数适合简单,一次性的函数场景,如键排序,映射转换等.复杂逻辑仍应该使用def定义具名函数从而可以提升代码可读性.不仅如此,lambda函数也可以用于固定函数参数,如

```
def dis(x,y):
    return abs(x-y)
dis10=lambda x:dis(10,x)
```

但是这样代码可读性较差,Python的functools模块中提供了partial方法,可以将多参函数固定参数使得转换为少参函数,从而提升代码复用效率.

```
from functools import partial
distance10=partial(dis,10)
print(distance10(2))
```

这里我们没有指定固定的参数位置,则默认为第一个参数.我们也可以直接指定固定某个函数参数如

```
from functools import partial
def power(base, exponent):
    return base ** exponent
square = partial(power, exponent=2)
cube   = partial(power, exponent=3)
```

从上面计算平方和和立方和的代码中,我们可以意识到Python代码的一个重要机制,Map-Reduce模式.我们可以将计算拆解为两个步骤,第一个是Map过程,对每个元素都应用函数,其可以实现分布式并行计算;第二个则是Reduce过程,则是基于前一步的基础上,将结果以某种方式累积成一个值.我们将两个步骤显式拆开,如下所示:

```
def map(func, values):
    return [func(x) for x in values]
def reduce(func, values, initial=0):
    result = initial
    for n in values:
        result = func(n, result)
    return result
nums = [1, 2, 3, 4]
result = reduce(lambda x, y: x + y, map(lambda x: x * x, nums))  # 30
print(result)
```

Python中存在内置函数map,其可以将函数直接作用在序列值.需要注意的是,map函数返回的实际上是一个迭代器,而不是列表,我们需要强制类型转换将结果变为列表.

```
print(list(map(square, [1, 2, 3])))
print(list(map(cube,   [1, 2, 3])))
```

4 Closures

闭包指的是如果内部函数作为结果被返回,那么这个内部函数就被称之为闭包,闭包会保留函数后续正常运行所需的所有变量的值.为了实现这一点,对外部变量(绑定变量)的引用会随函数一并携带.闭包通过closure属性保存对外部变量的引用.

```
def add(x,y):
    def do_add():
        print(f'{x}+{y}->{x+y}')
    return do_add
a=add(3,4)
print(a)# <function add.<locals>.do_add>
print(a.__closure__) # (<cell: int object>, <cell: int object>)
print(a.__closure__[0].cell_contents) # 3
print(a.__closure__[1].cell_contents) # 4
a()
```

每个cell对象会封装一个被捕获的变量值,查看具体的存储结果需要调用cell_contents来查看,并且closure属性是一个元组.

需要注意的是闭包只会捕获函数体内实际引用的变量,对于未使用的变量则不会被保留.

```
def new_add(x,y):
    result=x+y
    def get_result():
        return result
    return get_result
b=new_add(3,4)
print(b.__closure__)
print(type(b.__closure__))
print(b.__closure__[0].cell_contents)
```

上面并不会保留x和y的值,而只会保留result的值,因为内部函数只需要使用result变量.闭包的变量是不允许随意修改的,但是我们如果期待闭包变量可变,那么就需要使用nonlocal声明变量,这样的话,闭包就可以持有可变的内部状态,功能类似对象或类.nonlocal允许内部函数修改外层作用域的变量,使闭包具有状态保持能力.

```
def counter(n):
    def incr():
        nonlocal n
        n+=1
        return n
    return incr
c=counter(10)
print(c())
print(c())
```

闭包是Python的核心特征之一,常见应用包括:替代求值(延迟求值),回调函数,代码生成(宏).我们在此给出上述应用场景的相关代码实现.

```
def delayed_computation(x,y):
    print('Prepared Done')
    def compute():
        print("Performing computation")
        return x+y
    return compute
result_func=delayed_computation(10,20)
print(result_func())
```

上面的代码是延迟求值的相关实现.延迟计算直到真正需要的时候才会执行.

```
def make_callback(name,value):
```

```
    def callback():
        print(f'Callback {name}: value = {value}')
    return callback
cb1=make_callback('handler1',100)
cb2=make_callback('handler2',200)
cb1()
cb2()
def create_event_handler(event_name,handler_data):
    def handler(event):
        print(f"Event '{event_name}' triggered")
        print(f"Data: {handler_data}")
        print(f"Event details: {event}")
    return handler
click_handler=create_event_handler("click",{'count':0})
click_handler({'x':100,'y':200})
```

回调函数是因为闭包携带上下文信息,适合用作回调.

```
def make_multiplier(factor):
    def multiplier(x):
        return x*factor
    return multiplier
double=make_multiplier(2)
triple=make_multiplier(3)
percent=make_multiplier(0.01)
print(double(10))
print(triple(10))
print(percent(10))
def make_formatter(prefix, suffix):
    """生成一个带前缀和后缀的格式化函数"""
    def formatter(text):
        return f"{prefix}{text}{suffix}"
    return formatter
bold = make_formatter("**", "**")
italic = make_formatter("*", "*")
quote = make_formatter("> ", "")
print(bold("Hello"))    # **Hello**
print(italic("World"))  # *World*
print(quote("Note"))    # > Note
```

闭包用于生成定制化的函数

5 Errors

异常捕获的语法块结构如下所示

```
try:
    pass
except RuntimeError as e:
    pass
```

在函数中我们应该尽可能处理那些可以被恢复且具有恢复意义的异常.对于那些不好处理的异常,我们应该让他继续向函数外抛出异常,因为这类不好处理的异常通常表示更为复杂的问题.

```
def read_csv(filename):
    f=open(filename)
    for row in csv.reader(f):
        try:
            name=row[0]
            shares=int(row[1])
            price=float(row[2])
        except ValueError as e:
            print('Bad row:',row)
            continue
```

上面我们只针对ValueError进行了处理,如果我们发现出现了FileNotFoundError,也就是文件不存在的异常,我们实际上在程序层面是没有合理的恢复手段,所以我们需要将其抛出交由程序运行者处理.对于异常处理而言,我们并不希望捕获所有的异常,除非是我们报告或记录了真正的异常信息,如

```
try:
    pass
except Exception as e:
    print("Sorry, it didn't work.")
    print("Reason:",e)
```

如果我们不记录不报告真实异常,极易造成代码的无法维护.但是进一步,如果我们完全忽略代码异常的处理而强行推进程序,那么极易造成严重的程序事故.

在异常处理中,我们可能会希望记录抛出位置,或者对异常信息进行处理后重新抛出异常,我们可以在except语法块中重新调用raise从而抛出相同的异常.

```python
try:
    pass
except Exception as e:
    print("Sorry, it didn't work.")
    print("Reason:",e)
    raise
```

这样会将相同的异常再次向外抛出,需要强调的是,如果raise后面不接异常错误并且在except语法块之外,程序是会报错的.这样的操作适合于对异常进行处理后仍允许其向外传播.我们可以通过将异常包装成另一个异常的方式,来保留原始异常信息并且创建新的异常链.如下所示

```python
class TaskError(Exception):
    """自定义任务异常"""
    pass
def process_data(data):
    """底层处理函数"""
    if not data:
        raise ValueError("数据不能为空")
    return data.upper()
def run_task(data):
    """上层任务函数"""
    try:
        result = process_data(data)
        return result
    except Exception as e:
        # 保留原始异常信息，创建新的异常链
        raise TaskError('It failed') from e
try:
    run_task("")
except TaskError as e:
    print(f"捕获到: {e}")
    print(f"原始异常: {e.__cause__}")
```

例如上面的程序最初的异常是ValueError,在run_task函数中我们将其包装为TaskError异常.但他依旧保留了原始异常信息,我们可以通过__cause__属性来调用他.这里的from就表示从原有的异常中继承包装抛出一个新的异常信息.这里异常处理中还存在一部分资源管理的问题,异常处理中仍然需要进行合理的资源管理,不然的话,就可能导致文件描述符泄漏,死锁或其他问题.比较经典的方法可以考虑finally语法块,其表示无论异常是否发生都会运行.

```
try:
    pass
except Exception as e:
    pass
finally:
    pass
```

现代一点的方法则是使用with语法块来自动管理相关的资源.

```
with open(filename) as f:
    try:
        pass
    except Exception as e:
        pass
```

应用程序应定义自己的异常,并且我们应该将Python内置异常保留给编程错误.应捕获而不是主动抛出异常.

在C/C++中,我们可以通过返回码的方式来设置函数的返回状态.但在Python中尽可能不要使用返回码,是因为返回码不是Python中表示错误得到标准方式,调用者常会遗忘检查,导致程序稍后因其他原因崩溃.使用logging模块记录诊断信息,

```
import logging
log = logging.getLogger(__name__)

def read_data(filename):
    pass
    try:
        name = row[0]
        shares = int(row[1])
        price = float(row[2])
    except ValueError as e:
        log.warning("Bad row: %s", row)
        log.debug("Reason: %s", e)
```

这种记录方式通常会比print函数更合适.我们之前也已经介绍过logging模块,在此不再赘述.

6 Testing and Debugging

Python的动态特性提供了Python的很多方便的特性,但也使得代码测试对大多数应用至关重要,发现bug的唯一方法是运行并且确保覆盖其所有功能.断言是运行时检查,如果不符合断言则会报错,抛出AssertionError.

```
def add(x, y):
    ''' Adds x and y '''
    assert isinstance(x, int)
    assert isinstance(y, int)
    return x + y
```

断言不应该用于检查用户输入而应用于验证程序不变量(如必须始终成立的内部条件).运行失败则表示编程错误,那么可以基于这个断言进行责任规划(如归咎于调用者).同样我们虽然写了断言,但是可能在运行中需要忽略断言的语句,所以我们可以禁用断言,用如下的调用Python解释器方式,

```
python -O Ch5_Ex5.py
```

Python的内置测试模块为unittest模块,其可以应用于标准库使用,广泛应用于其他项目.我们定义一个测试类,其必须继承自unittest.TestCase,并定义其中的测试方法,代码如下所示

```
import simple
import unittest
class TestAdd(unittest.TestCase):
    def test_simple(self):
        # 用简单整数参数测试
        r = simple.add(2, 2)
        self.assertEqual(r, 5)
    def test_str(self):
        # 用字符串测试
        r = simple.add('hello', 'world')
        self.assertEqual(r, 'helloworld')
```

这里的每个测试方法必须要以test开头,这样的话才会被测试模块自动调用检测.我们稍微举例一下测试中的特殊断言,

```
# 断言 expr 为 True
self.assertTrue(expr)
# 断言 x == y
self.assertEqual(x, y)
# 断言 x 与 y 近似相等
self.assertAlmostEqual(x, y, places)
# 断言会抛出异常
with self.assertRaises(SomeError):
    pass
```

如果我们需要运行unittest测试,那么应该添加以下的代码,

```
if __name__ == '__main__':
    unittest.main()
```

unittest模块对于大型应用可能会变得十分复杂,并且unittest在测试运行,结果收集方面有大量选项.

Working with Code

1 Variable Arguments

函数参数表示函数可以操作传入的参数,其中的基本传入方式有如下两种:

```
func(1,2,3) # 位置参数
func(x=1,y=2,z=3) # 关键字参数
```

我们可以混合使用这两种参数传入风格,但是位置参数必须始终在关键字参数前面传入,而且需要遵守每个参数都必须有且仅有一个值,如

```
def fun(x,*args):
    pass
func(1,x=1) # 这会报错
```

混合使用参数风格增加了调用的灵活性,尤其是当函数参数比较多的时候,关键字参数能够提高代码的可读性.

可变数目参数则是指可以接受任意数目的位置参数或关键字参数.先介绍可变位置参数,代码结构如下,

```
def func(x,*args):
    print(x)
    print(args)
```

其表示至少需要一个参数传入给x,而其余参数则会以元组的方式传入给args.同样可以接受相应的关键字参数,代码结构如下

```
def foo(x,y,**kwargs):
    print(x)
    print(y)
    print(kwargs)
foo(2,3,flag=True,mode='fast',header="debug")
```

其余的关键字参数则会以字典的形式传入给kwargs.二者还可以合并使用,如

```
def func(*args,**kwargs):
    pass
```

这表示这个函数接受任意数目的位置参数或关键字参数,唯一需要注意的是,位置参数必须在关键字参数前出现.这个的用处一般是用于编写包装器或者将函数的参数转移给另一个函数.*args和**kwargs是编写装饰器的关键,因为它允许包装函数接受被包装函数所能接受的所有参数,而不需要知道他的具体参数签名.这里的args和kwargs只是一个约定俗成的名字,可以根据自己喜好更换,但不是很建议.

我们前面提到可变位置参数传入函数内会以元组的形式存储,可变关键字参数则是以字典的形式存储,所以其实元组和字典也是可以通过解包的方式以各个独立参数的形式传入函数的.元组是通过*做解包,而字典则是通过**做解包.这其实就是可变参数的逆过程,我们在定义函数的时候用*来收集函数参数,而在调用时则用*对元组进行解包.这一操作一般用在配置管理,将配置项提前存储在字典,然后调用函数一次性解包传入,使得代码更加整洁.

2 Scoping Rules

程序将值赋给变量的时候,一般是在两个大作用域下完成的,全局作用域和局部作用域.

```
x=value # Global variable
def func:
    y=value # Local variable
```

所以简单来看全局作用域就是在函数外赋值,局部作用域则是在函数内赋值.基本上所有的语句都在这两个作用域中执行.全局作用域始终是定义函数的模块,局部作用域则函数私有的,或者与全局作用域的相同字典.变量的查找顺序是遵循LEGB.需要注意的是,Python中其实没有真正的全局变量只有模块级变量,因为每个python文件都是一个模块,具有自己的全局命名空间.

如果我们希望在函数中修改全局变量,如果只是用普通的x=val,那么会创建一个同名的局部变量,从而覆盖了全局变量的读取.如果需要修改全局变量,我们需要使用global声明,并且需要在使用前调用,而且仅对于要被修改的全局变量是必须的,因为全局变量已经可读.

```
x=1
def func():
    global x
    x=2
```

在函数中可以直接读取全局变量,而不用global声明,只有我们希望修改全局变量,才应该对其做global声明.但是如果全局变量是可变的,例如x=[],如果我们只是做append之类的原地操作,那么是不需要做global声明的.因为x指向的是列表对象,原地操作则是不会修改变量的指向,我们只是填充列表对象,因此不需要做声明,只有在重新绑定才会需要声明global声明.我们需要注意的是尽量不要在函数内部修改全局变量的状态.

globals()函数可以以字典形式返回全局作用域的内容;locals()函数可以以字典形式返回局部作用域的内容.可以借助这两个函数检查代码实现.需要注意的是locals和globals函数在模块中调用,那么他们的输出会是一样的.

```
x=1
def func():
    test=1
    print(globals())
    print(locals())
print(globals())
print(locals())
```

内置函数会存在在builtins module.如果查找不到相应的变量,则是会去builtins module中查找.并不建议修改builtins modules.

我们给出一个比较技巧性的Python编程技巧,

```
class Stock(Structure):
    _fields=('name','shares','price')
    @staticmethod
    def _init():
        locs=sys._getframe(1).f_locals # Get callers local variables
        self = locs['self']
        for name, val in locs.items():
            if name == 'self': continue
            setattr(self, name, val)
    def __init__(self,name,shares,price):
        self._init()
```

我们主要介绍其中的sys._getframe(1).f_locals的作用.sys._getframe(n)是一个用于内省的函数,用于访问当前的调用栈.n=0代表当前函数,也就是_init方法本身;n=1代表调用当前函数的那个函数,也就是__init__方法,所以此时获取的是__init__执行的时候的栈帧对象.f_locals则是栈帧对象的一个属性,返回一个字典,包含了该函数作用域的所有局部变量,上面的调用则是大体上可能会返回如下结果,

```
{
    'self': <Stock 对象实例>,
    'name': 'ACME',
    'shares': 50,
    'price': 91.1
}
```

所以locs的赋值过程相当于就像是_init方法读取了__init__方法的内部所有变量.

3 Function objects

当我们完成了函数的定义的时候,函数其实会变成一个可以操作的对象,他可以用于变量赋值,存取容器以及传入参数.函数的第一行可以是字符串,并且会被默认为文档字符串,其会存储在函数的__doc__属性.help函数可以查看函数的文档字符串,建议用三引号编写多行文档字符串.

```
def func(a, b):
    'This function does something.'
    pass
print(func.__doc__)
help(func)
```

在函数定义的时候,可以添加一些注释.参数和返回值是允许被添加注释的,如

```
def func(a:int,b:int)->int:
    pass
```

我们可以在函数的__annotations__属性中查看相应的注释.但这类注释什么都不会做,他只是提供变量期待的类型信息,不会强制要求满足类型.我们可以给函数添加任意形式的属性,他会存放在函数的__dict__属性中,这可以使函数具有静态变量的状态.这一特性一般用于给函数标记属性,例如在Web开发的时候用来标注某个函数需要认证,则可使用如下的代码.

```
func.requires_auth=t=True
```

我们可以检查函数的几乎所有属性.我们罗列如下

```
print(func.__name__)
```

用于获取函数的名称,以字符串形式返回

```
def func(a,b=1):
    pass
print(func.__defaults__)
```

获取函数位置参数的默认值,以元组的形式返回,如果没有,那就返回None.

```
def func(a,*,b=1):
    pass
print(func.__defaults__)
```

获取函数中仅关键字参数的默认值,以字典的形式返回,如果没有,那就返回None.

```
print(func.__code__)
```

他会返回一个code对象,获取函数的代码对象,包含函数编译后的字节码以及相关信息.

```
print(func.__code__.co_argcount)
```

他会计算函数定义的位置参数的数量,不包括*args和**kwargs.

```
print(func.__code__.co_varnames)
```

获取函数中所有局部变量名的元组,包括传入的参数名和内部定义的变量.如果我们希望查找函数的其他属性,那么可以用dir函数来查看函数的具体属性.

Python中提供了inspect模块,用于在运行时获取对象的内部信息.

```
import inspect
def func(a,b=10,*args,**kwargs):
    pass
# 获取函数签名
sig=inspect.signature(func)
print(sig)
# 获取参数信息
for name,param in sig.parameters.items():
    print(f"{name}: {param.default}")
```

这是inspect最为核心的功能之一.inspect.signature获取可调用对象(如函数)的签名对象.它包含了函数的参数名,默认值以及类型注解等信息.sig.parameters.items遍历签名中的所有参数.param.default获取参数的默认值,如果参数没有默认值,那么就会返回<class 'inspect._empty'>

```
import inspect
import sys
# 定义测试对象
def my_function(x, y):
    return x + y
class MyClass:
    def instance_method(self):
        pass
    @classmethod
    def class_method(cls):
        pass
    @staticmethod
    def static_method():
        pass
my_instance = MyClass()
# 内置模块和函数
import math
# 执行类型检查
print("=" * 50)
print("inspect 类型检查函数验证")
```

```python
print("=" * 50)
print(f"\n1. inspect.isfunction(my_function):")
print(f"   结果: {inspect.isfunction(my_function)}")
print(f"   说明: my_function 是普通函数 ✓")
print(f"\n2. inspect.isclass(MyClass):")
print(f"   结果: {inspect.isclass(MyClass)}")
print(f"   说明: MyClass 是类 ✓")
print(f"\n3. inspect.ismodule(sys):")
print(f"   结果: {inspect.ismodule(sys)}")
print(f"   说明: sys 是模块 ✓")
print(f"\n4. inspect.ismethod(my_instance.instance_method):")
print(f"   结果: {inspect.ismethod(my_instance.instance_method)}")
print(f"   说明: 绑定到实例的方法是方法 ✓")
print(f"\n5. inspect.isbuiltin(len):")
print(f"   结果: {inspect.isbuiltin(len)}")
print(f"   说明: len 是内置函数 ✓")
# 补充对比测试
print("\n" + "=" * 50)
print("补充对比测试")
print("=" * 50)
print(f"\ninspect.isfunction(MyClass.instance_method):")
print(f"   结果: {inspect.isfunction(MyClass.instance_method)}")
print(f"   说明: 未绑定的类方法是函数")
print(f"\ninspect.isfunction(my_instance.static_method):")
print(f"   结果: {inspect.isfunction(my_instance.static_method)}")
print(f"   说明: 静态方法被视为函数")
print(f"\ninspect.ismethod(MyClass.class_method):")
print(f"   结果: {inspect.ismethod(MyClass.class_method)}")
print(f"   说明: 类方法也是方法")
print(f"\ninspect.isbuiltin(math.sqrt):")
print(f"   结果: {inspect.isbuiltin(math.sqrt)}")
print(f"   说明: math.sqrt 是内置函数")
# 批量检查示例
print("\n" + "=" * 50)
print("批量类型检查示例")
print("=" * 50)
objects = {
    '普通函数': my_function,
    '类': MyClass,
    '实例方法': my_instance.instance_method,
    '类方法': MyClass.class_method,
    '静态方法': my_instance.static_method,
    '内置函数': len,
    '模块': sys,
```

```
    '字符串': "hello",
    '整数': 42,
}
for name, obj in objects.items():
    checks = {
        'isfunction': inspect.isfunction(obj),
        'ismethod': inspect.ismethod(obj),
        'isclass': inspect.isclass(obj),
        'ismodule': inspect.ismodule(obj),
        'isbuiltin': inspect.isbuiltin(obj),
    }
    result = ', '.join([k for k, v in checks.items() if v])
    print(f"{name:10s}: {result if result else '无匹配类型'}")
```

inspect模块中存在很多is开头的函数,用于验证对象是否为所需要的类型.

```
# 获取源代码（字符串）
source = inspect.getsource(func)
# 获取源文件路径
file_path = inspect.getfile(func)
# 获取代码行号
line_no = inspect.getsourcelines(func)[1]
print(source)
print(file_path)
print(line_no)
```

它可以用于反查代码的物理位置和具体内容.getsource返回函数定义的源代码字符串.只能用于Python编写的函数,不能用于内置函数或交互式命令行中定义的函数.getfile返回定义该对象的文件路径.getsourcelines返回一个元组(source_lines,start_line_no).这里的函数必须是用户自定义的函数.

```
sig = inspect.signature(func)
args = (1, 2)
kwargs = {'c': 10}
bound = sig.bind(*args, **kwargs)
for name, val in bound.arguments.items():
    print(name, '=', val)
```

在不实际执行函数的情况下,模拟参数传递过程,检查参数是否匹配.sig.bind(*args, **kwargs):尝试将提供的参数绑定到函数签名上.其作用是验证参数,如果参数数量不对或名称错误,会抛出TypeError,而不会执行函数体;获取最终参数映射,bound.arguments是一个有序字典,显示了每个参数名最终对应的值,包括默认值填充.

4 Eval/Exec

eval(expr)用于计算一个表达式的值.

```
x=10
expr='3*x-2'
print(eval(expr))
```

exec(code)用于执行任意的函数语句.

```
exec('for i in range(10): print(i)')
```

这些代码会在globals或locals中执行.区别在于eval只能处理单个表达式并且返回值,exec可以处理任意语句块(如循环,定义类),但不返回值,会返回None.动态编程指允许程序在运行时生成并执行代码.注意在exec的执行不会影响局部作用域.

```
def func():
    x=15
    exec('x=10;print(x)')
    print(x)
func()
```

eval和exec的语法结构如下:

```
eval(expr[,globals[,locals]])
exec(code[,globals[,locals]])
```

所以这样我们可以用exec修改局部变量的结构如下所示:

```
def foo():
    x=15
    loc=locals()
    exec('x=10;print(x)',globals(),loc)
    x=loc['x']
    print(x)
```

在Python函数内部为了速度,优化了局部变量的存储,exec试图修改局部字典,但无法同步到优化的局部变量存储.从locals读取结果,应该避免这样的代码错误,不可靠且难以理解.对于eval/exec,极小心使用这类功能,存在不可对输入潜在安全问题.

5 Callable Object

对于函数而言,我们可以定义自己的对象来模拟Python的函数(如callables),但必须实现call特殊方法.故而func调用的本质是调用func.__call__().

```
class Memoize:
    def __init__(self,func):
        self._cache={}
        self._func=func
    def __call__(self,*args):
        if args in self._cache:
            return self._cache[args]
        r=self._func(*args)
        self._cache[args]=r
        return r
    def clear(self):
        self._cache.clear()
@Memoize  # 等同于 fib = Memoize(fib)
def fib(n):
    if n < 2:
        return n
    return fib(n-1) + fib(n-2)
# 调用时完全像普通函数
print(fib(10))
fib.clear()  # 清除缓存，这是普通函数做不到的
```

MetaProgramming

1 Decorators

在编写程序中存在大量代码重复的问题,其在编写过程会十分枯燥无聊,并且后续难以维护.元编程涉及编写操作其他代码的代码问题.其相应的应用示例为宏(Marco),包装器(wrapper),切面(aspect).因此其本质上是对代码进行操作.元编程的核心动力是DRY原则(Don't Repeat Yourself). 当发现存在大量代码结构类似的情况,则可以用元编程.与C/C++的宏在编译前处理文本,Python的元编程是在运行的时候操作对象(func,class).其会更加的灵活,更加的动态.装饰器是一个创建另一个函数包装器的函数.包装器则是一个全新的函数,但是他的工作原理和原有函数是完全相同的,接受相同的函数参数可以返回相同的返回值,但是可以增加一些额外的操作.装饰器本质上是接受函数作为参数并返回新函数的高阶函数.理想的装饰器应该是对被装饰器的调用者是透明的,也就是即使换成被装饰了的函数,其调用和返回也不应该导致代码出错.而额外的输出则是由包装器函数实现,但仍然依靠原始函数获得结果.

```
def add(x,y):
    return x+y
def logged(func):
    def wrapper(*args,**kwargs):
        print('Calling',func.__name__)
        return func(*args,**kwargs)
    return wrapper
log_add=logged(add)
print(log_add)
print(log_add(3,4))
```

这里我们发现这个logged函数的内部函数wrapper内部调用了func,所以其实wrapper函数就是前面介绍过的闭包函数,这样可以使得wrapper能够记住他要包装的那个专有函数.上面的代码中需要为每个函数编写log_func=logged(func),这要求我们额外多写很多的重复代码.

```
@logged
def add(x,y):
    return x+y
# add=logged(add)
```

这相当于是将add的指向函数进行了修改,从原始函数修改到包装器函数.当我们使用装饰器替代函数,会添加一些函数功能,这也就是所谓的装饰.装饰器不修改原函数,而是做一个包装.原函数不变,只是调用路径发生了改变.常用的装饰器如下

```
@staticmethod
@classmethod
@property
name=decorator(name)
```

装饰器可以在任意时间定义一个涉及函数操作的高阶函数,其主要应用是调试诊断,避免代码重复,启用/禁用可选功能.

2 Advanced Decorators

高阶装饰器指的是多重装饰器,装饰器和元数据和带参数的装饰器.装饰器可以多次嵌套使用,

```
@foo
@bar
@spam
def func():
    pass
# func=foo(bar(spam(func)))
```

执行顺序:装饰器是从下向上的应用;调用顺序:调用函数时,从外向内调用.但是多层嵌套会使得调试更困难,错误堆栈也会更长.但是这个潜逃需要十分注意装饰器的上下顺序,不同的顺序可能会导致结果的不同.当定义一个函数,同时会存储一些额外信息(名称,文档字符串等)

```
def add(x, y):
    'Adds x and y'
    return x + y
print(add.__name__)
print(add.__doc__)
help(add)
```

内省基础:这些属性(name,doc,module)是工具依赖的基础.装饰器则不会保持这些元数据,因为装饰器实际指向了另一个函数,所以出错的时候函数堆栈跟踪的是包装器函数.

```
@logged
def add(x, y):
    'Adds x and y'
    return x + y
print(add.__name__)
print(add.__doc__)
```

我们可以手动加上复制函数的元数据的代码.

```
def logged(func):
    def wrapper(*args,**kwargs):
        print('Calling',func.__name__)
        return func(*args,**kwargs)
    wrapper.__name__=func.__name__
    wrapper.__doc__=func.__doc__
    return wrapper
@logged
def add(x, y):
    'Adds x and y'
    return x + y
print(add.__name__)
print(add.__doc__)
```

当然也可以用如下方式自动的复制函数元数据,

```
from functools import wraps
def logged(func):
    @wraps(func)
    def wrapper(*args,**kwargs):
        print('Calling',func.__name__)
        return func(*args,**kwargs)
    return wrapper
@logged
def add(x, y):
    'Adds x and y'
    return x + y
print(add.__name__)
print(add.__doc__)
```

这里会自动将原函数的元数据复制到wrapper函数上去.从上面的代码来看,wraps装饰器是能够接受参数的,所以我们接下来介绍带参数的装饰器.

```
@decorator(x,y,z)
def func():
    pass
# func=decorator(x,y,z)(func)
```

装饰器函数必须返回一个函数,这个函数会用于创建包装器函数.对于这类带参数的装饰器,实际上有三层的参数嵌套,外层接受装饰器参数,中层接受被装饰函数,内层接受调用参数.其调用链由decorator(x,y,z)返回中层函数,其会被被装饰函数调用.其外部函数用于接受参数,内部函数则是标准的装饰器代码.因此带参数的装饰器更加通用,并且也可以将带参数的装饰器实例化为无参数的装饰器变量.

```
def logmsg(message):
    def logged(func):
        @wraps(func)
        def wrapper(*args,**kwargs):
            print(message.format(name=func.__name__))
            return func(*args,**kwargs)
        return wrapper
    return logged
@logmsg('You called {name}')
def add(x,y):
    return x+y
print(add(3,4))
logged=logmsg('You called {name}')
@logged
def add(x,y):
    return x+y
```

```
print(add(3,4))
```

3 Class Decorator

装饰器不仅可以应用在函数上,还可以应用在类定义.

```
@decorator
class Myclass:
    def foo:
        pass
# 上面的相当于Myclass=decorator(Myclass)
```

所以也就是装饰器操作或包装一个类.类装饰器一般常用于自动注册类,验证类结构或者批量修改类方法.大多数类装饰器会检查类定义或做一些特殊处理.类装饰器的语法结构如下所示

```
def decorator(cls):
    # Do something with cls
    # Return the original class back
    return cls
```

这里我们可以发现类装饰器通常不会修改原始类并且返回.函数装饰器通常会返回一个新的函数,而类装饰器通常直接修改传入的类对象cls并返回相应的类对象.

```
def logged_getattr(cls):
    # Get the original implementation
    orig_getattribute=cls.__getattribute__
    # Replacement method
    def __getattribute__(self,name):
        print("Getting:",name)
        return orig_getattribute(self,name)
    # Attach to the class
    cls.__getattribute__=__getattribute__
    return cls
@logged_getattr
class MyClass:
    def __init__(self,name,size):
        self.name=name
        self.size=size
    def foo(self):
        pass
s=MyClass(10,9)
print(s.name)
print(s.size)
s.foo()
```

这里我们提供了一个类装饰器的例子,他的作用是在调用类成员的时候,会宣告查看的类成员名称.

现代的Python3.6以上的版本提供了__init_subclass__方法,他其实比前面介绍的类装饰器更好用,所以我们在此加以解释一下.通常父类不知道子类继承的情况,但是定义了__init_subclass__的类,是可以在类被继承的时候察觉到子类的存在,并且对子类进行自动化的操作.

```
class PluginBase:
    subclasses = []
    @classmethod
    def __init_subclass__(cls, **kwargs):
        super().__init_subclass__(**kwargs)
        # 只要有人继承我，我就把它存到列表里，实现自动注册
        cls.subclasses.append(cls)
class VideoPlugin(PluginBase): pass
class AudioPlugin(PluginBase): pass
print(PluginBase.subclasses)
```

这里的cls表示刚刚创建的子类,这其实也是表示父类可以获取子类的信息.

4 Type

Python中的任意值都有自己的类型,并且类型其实也是一个可调用对象,我们可以调用他来创建一个同类型的对象.

```
x=43
print(type(x))
s='Hello'
print(type(s))
items=[1,2,3]
print(type(items))
a=list()
print(a)
b=tuple(items)
print(b)
```

不仅如此,我们自定义创建的类在Python中也是类型的一种.所以类可以自定义一个新类型,他的调用方式和内置类型一样.

```
class Spam:
    pass
s=Spam()
print(type(s))
print(type(Spam))
```

类是创建的实例的类型,类也是创建实例的可调用对象.我们对于类调用type函数可以得知,类的类型其实是type类.换言之,类其实是type类的一个实例.而type类本身则是由他们自己表示,所以我们定义自己的类其实在Python内部是由type类做的处理,因此我们称type类为元类,简单来说他就是所有类的类.

我们对类的定义过程进行一个解构,首先我们常见的类定义方式如下所示,

```
class Spam(object):
    def __init__(self,name):
        self.name=name
    def yow(self):
        print("Yow!",self.name)
s=Spam('Test')
s.yow()
```

但实际上我们可以将他分成三步:第一步我们先定义一些需要的函数方法,

```
def __init__(self,name):
    self.name=name
def yow(self):
    print("Yow!",self.name)
```

第二步我们则是构造方法字典,他的键名是方法名,而他的值则是上面定义的函数对象,两者可以不同,但是不建议,因为会使得程序维护困难,

```
methods={'__init__':__init__,'yow':yow}
```

最后我们调用type函数来构造一个类

```
Foo=type('Foo',(object,),methods)
type(clsname,(Base,),methods)
```

这里的第一个参数clsname表示想要创建的类名,第二个参数则表示需要的基类元组,第三个参数则是方法字典.

在此基础上我们给出一个利用type构造函数和命名空间字典来创建类的方式.首先,我们需要准备命名空间,语法如下所示,

```
__dict__=type.__prepare__('Spam',(object,))
```

在正常的class语句的开始,Python会先调用元类的__prepare__方法,其作用是创建一个用于保存类成员(属性和方法)的字典,他通常返回一个空的dict,但在某些复杂的元类编程中,会返回一个OrderedDict或其他特殊映射.这里传入基类的原因,一是用来检查父类中是否有自定义的元类,如果有,那么Python就会调用其元类的方法而不是type的默认方法,二则是默认情况下会返回一个字典,但是某些高级模块会要求返回有序字典,这样的话,可以保证顺序.

第二步用于填充类信息

```
__dict__['__qualname__'] = 'Spam'
__dict__['__module__'] = 'modulename'
```

这里其实就是在模拟Python编译器自动完成的工作,__qualname__表示类的限定名,__module__表示类的模块名,如果不手动设置,那么手动创建的类可能会丢失这些元数据.

第三步是执行类体代码也是最为关键的一步

```
body='''
def __init__(self,name):
    self.name=name
def yow(self):
    print("Yow!",self.name)
'''
exec(body, globals(), __dict__)
```

body表示一个包含函数定义的字符串,exec则是在__dict__这个命名空间里面执行字符串代码.执行后,__dict__中会多出两个键:__init__和yow,它们分别对应定义的函数对象.

第四步,形成真正的类

```
Spam = type('Spam', (object,), __dict__)
```

整体代码如下

```
body='''
def __init__(self,name):
    self.name=name
def yow(self):
    print("Yow!",self.name)
'''
__dict__=type.__prepare__('Spam',(object,))
print(__dict__)
__dict__['__qualname__'] = 'Spam'
__dict__['__module__'] = 'modulename'
exec(body,globals(),__dict__)
print(__dict__)
Spam = type('Spam', (object,), __dict__)
```

5 MetaClass

创建类的类称之为元类,type则是元类的一个实例.Python提供了一个Hook,允许你覆盖类创建步骤.我们在创建类的时候可以选择使用不同于type的元类.使用这个,你可以完全自定义类在创建时发生的事情.其相关的语法调用如下所示,

```
class Spam(metaclass=type):
    def __init__(self,name):
        self.name=name
    def yow(self):
        print("Yow!",self.name)
s=Spam('Test')
s.yow()
```

我们发现在类创建的时候提供了一个元类参数,其提供一个metaclass关键字参数,在此我们可以设置用于创建类对象的类,也就是所谓的元类选择.只是默认来说我们直接使用type元类,因此我们平时见到的元类并不多,当然我们也可以设置成自己定义的元类.上面提到的用法其实是Python3的语法,在Python2中,我们需要用如下的语法结构,

```
class Spam:
    __metaclass__=type # Python2 only
    def __init__(self,name):
        self.name=name
    def yow(self):
        print('Yow!',self.name)
```

如果我们不显式的设置元类,那么Python会直接使用父类的元类创建.

```
class Spam(object):
    def __init__(self,name):
        self.name=name
    def yow(self):
        print('Yow!',self.name)
```

我们提供一种自定义新元类的方式,我们可以通过继承type类并且重定义类的方法,如__new__,__prepare__等等.

```
class mytype(type):
    @staticmethod
    def __new__(meta,name,bases,methods):
        print('Creating class:',name)
        print('Base classes:',bases)
        print('Methods:',list(methods))
        return super().__new__(meta,name,bases,methods)
```

基于上面定义的元类,我们可以定义一个新的根对象,

```
class myobject(metaclass=mytype):
    pass
```

为了使用我们上面定义的新元类,我们可以通过继承根类对象来定义自己的类,元类的new方法是用于创建类对象本身,而类的new方法则是创建一个未初始化的类实例对象.

```
class Spam(myobject):
    def __init__(self, name):
        self.name = name
    def yow(self):
        print("Yow!", self.name)
```

6 Application

元类允许改变类定义过程本身,设置类定义环境,改变实例创建.元类允许监控和操作类定义,他有四个比较关键的拦截点,分别如下所示:

```
# 在类定义开始之前调用
# 返回一个映射对象,通常是字典或有序字典,这个字典将作为类的命名空间,用于存储类定义过程中产生的属性和方法.
# 允许你控制类属性的存储顺序或收集定义时的元数据
type.__prepare__(name,bases)
# 在类主体代码执行完毕后调用,此时prepare返回的字典已经填满了类属性和方法
# 返回类对象本身
# 真正创建类对象的内存空间,如果返回的不是当前类的实例,那么不会调用元类的__init__
type.__new__(type,name,bases,dict)
# 在实例化类时调用, 他是实例创建过程中的第一步
# 返回实例对象
# 分配内存并返回一个未初始化的对象.如果new方法没有返回当前类的实例,那么init方法就不会被调用
cls.__new__(cls,*args,**kwargs)
# 在元类的new方法成功返回类对象后调用,标志类定义完成.
# 必须返回None
# 对创建好的类对象进行额外的配置或修改
type.__init__(cls,name,bases,dict)
# 在类的new方法返回实例对象后调用,此时实例已经存在,但是属性尚未赋值
# 必须返回None
# 初始化实例的属性
cls.__init__(*args,**kwargs)
# 元类方法,所属对象是元类,因为类是元类的实例,因此类可以被调用
# 当你执行Class()实例化对象的时候,会被调用;实际上就是会调用type.__call__
```

```
# 返回一个初始化完成的实例对象,他是实例化过程的控制器.默认的call函数内部逻辑是,先调用类的new方法创建实例,再调用类的init初始化实例,并返回实例.重写他可以控制实例化的整个过程.
type.__call__(cls,*args,**kwargs)
```

类定义方法的重复检查

```
class dupedict(dict):
    def __setitem__(self,key,val):
        assert key not in self, '%s duplicated' % key
        super().__setitem(key,val)
class dupemeta(type):
    def __prepare__(name,bases):
        return dupedict()
class A(metaclass=dupemeta):
    def bar(self):
        pass
    def bar(self):
        pass
```

对类中的每个方法都做装饰器操作,

```
def decorator(func):
    print('Decorating',func.__name__)
    def wrapper(*args,**kwargs):
        print('Calling',func.__name__)
        return func(*args,**kwargs)
    return wrapper
class meta(type):
    def __new__(cls,name,bases,methods):
        for key,val in methods:
            if callable(val):
                methods[key]=decorator(val)
        return super().__new__(cls,name,bases,methods)
```

实例创建

```
class meta(type):
    def __call__(cls,*args,**kwargs):
        print('Calling instance of',cls)
        return super().__call__(*args,**kwargs)
class Spam(metaclass=meta):
    def __init__(self, name):
        self.name = name
```

单例模式

```
class meta(type):
    _instance=None
    def __call__(cls,*args,**kwargs):
        if not cls._instance:
            cls._instance=super().__call__(*args,**kwargs)
        return cls._instance
class Spam(metaclass=meta):
    def __init__(self,name):
        print('Creating Spam')
        self.name=name
s=Spam('Test')
s1=Spam('Test1')
print(s1.name)
print(id(s)==id(s1))
```

Iterators,Generators and Coroutines

1 Iterator

迭代定义经常发生在循环遍历项的时候发生调用,代码如下所示,

```
a=[2,4,10,37,62]
for x in a:
    print(x)
```

这是一个十分常见的代码编程模式,常见于循环,列表推导式等.他的迭代语法原型为

```
for x in obj:
    # statements
```

他的底层语法结构为

```
_iter=obj.__iter__() # Get iterator object
while True:
    try:
        x=_iter.__next__() # Get next item
    except StopIteration: # No more items
        break
    # statements
```

所有能在for循环中遍历执行的对象,都应该在底层实现迭代协议,如__iter__方法(获取迭代对象),__next__方法(获取迭代的下一个对象).如果迭代器完成迭代,就会返回一个StopIteration.

```
x=list(range(10))
print(x)
it=x.__iter__()
print(it)
print(it.__next__())
print(it.__next__())
print(it.__next__())
```

自定义容器可以进行委托迭代.这是一种封装模式,如Portfolio类内部有一个列表,其不想暴露列表本身,但仍然想要在外部能够迭代调用.通过迭代委托的方式给内部对象,这样既能够保持了封装性,又可以获得迭代的便利性.

```
class Portfolio:
    def __init__(self):
        self.holdings=[]
    def __iter__(self):
        return self.holdings.__iter__()
p=Portfolio()
p.holdings.append(1)
p.holdings.append(2)
p.holdings.append(3)
for x in p:
    print(x)
```

生成器则是可以简化自定义迭代的流程,代码如下所示,

```
class Countdown:
    def __init__(self,n):
        self.n=n
    def __iter__(self):
        n=self.n
        while n>0:
            yield n
            n-=1
c=Countdown(5)
for i in c:
    print(i)
```

这里的yield关键字是生成器的核心,他和普通函数不同,其不会一次性返回所有结果并结束,而是应每次产生一个值并且停止运行.他的方便之处在于不需要手动管理__iter__方法和__next__方法.生成器函数的行为不同还在于他的返回会创建一个生成器对象,但不会开始运行函数,需要注意的是此时是没有产生输出值的.这是因为yield会出现一个惰性行为,计算和迭代推进只在需要的时候才会发生.函数只会在next函数调用的时候执行,yield产生一个值,然后会将函数挂起,函数一直到下一次调用next方法才会恢复执行.生成器具有记忆能力,当他挂起的时候,所

有局部变量都会被保存;下次恢复的时候,会从上次停下的地方开始,直到遇见next函数,才会重新停止函数执行.当生成器开始返回值的时候,也就代表迭代的结束.生成器函数实现了与for语句在列表,元组,字典和文件上的使用相同的底层协议.但是我们需要注意的是,生成器函数只能够使用一次,如果我们需要多次使用,应该需要再次定义.

```
c=Countdown(5)
for i in c:
    print(i)
for i in c:
    print(i)
```

生成器如果耗尽,那么就会抛出StopIteration,无法重置状态.这样可以确保数据流的单向性和内存高效性.因此,引入一个可重复使用的生成器类,只需要在其中创建一个__iter__方法,每次迭代都会重新创建一个新的生成器.

```
class FRANGE():
    def __init__(self,start,stop,step):
        self.start=start
        self.stop=stop
        self.step=step
    def __iter__(self):
        n=self.start
        while n<self.stop:
            yield n
            n+=self.step
f=FRANGE(0,2,0.35)
for x in f:
    print(x,end=' ')
print()
for x in f:
    print(x,end=' ')
print()
```

并且生成器可以用于转换为列表,元组和实现解包操作.

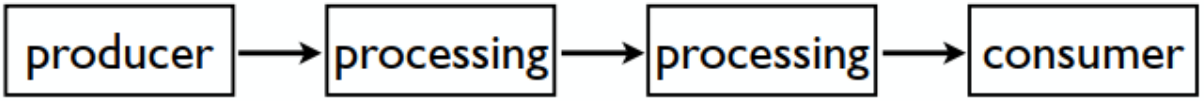
```
# s 是一个可重用的生成器类
print(list(s))
print(tuple(s))
name,shares,price=s
```

2 Producers & Consumers

生成器常用于生产者-消费者编程相关,其中yield生产值,for消费值.

```
# producers
import os
import time
def follow(filepath):
    f=open(filepath)
    f.seek(0,os.SEEK_END)
    while True:
        line=f.readline()
        if line=='':
            time.sleep(0.1)
            continue
        yield line
# consumers
for line in follow(filepath):
    print(line)
```

生成器天然适合作为生产者,因为可以按需产生数据,而非一次性将数据存入内存,适合于处理流式数据.利用生成器来构造数据处理管道.其中管道由初始数据生产者,中间数据处理模块和最终数据消费者构成.其结构如下,



生产者一般是生成器,当然也可以是列表或者其他的序列容器.yield可以将数据传入给管道.他的作用是负责数据源,不需要知道数据的后续处理方式,只需要负责生成数据.消费者一般是for循环,他会获取数据并且对他们进行操作,消费者是为了驱动整个管道的数据流运行.中间处理模块则是同时消费和生产数据,他会在管道中间修改数据流,并且可以过滤并丢弃数据.当我们用代码完成了管道的架构,此处不会立即运行程序,而只有在消费者开始迭代运行的时候,数据流才会开始流动,而且内存占用仅仅取决于单个阶段处理的数据量.

3 Coroutine Functions

在生成器中在,yield可以用作表达式.

```python
def match(pattern):
    print('Looking for %s' % pattern)
    while True:
        line =yield
        if pattern in line:
            print(line)
```

这里的yield出现在赋值语句右端的时候,他不再仅仅是产生值,而是等待接受外部发送进来的值,这开启了与程序外双向通信的可能性.上述的yield调用方式,就会形成一个协程函数,它定义了一个外界可以向其发送值的函数,发送的值由yield返回.协程允许数据流入函数内部,这与普通生成器(数据流出)相反,send方法发送的值会成为yield表达式的值并且完成赋值.

```python
m=match('python')
print(m)
m.send(None)
m.send('hello python')      # 输出: hello python
m.send('java is good')      # 无输出
m.send('I love!')    # 输出: I love python
m.close()
```

他的执行方式和生成器一样,调用协程函数也是什么都不会发生,只是返回一个生成器对象.只有在调用在send方法以后才会运行,所以协程函数也是惰性执行.调用函数只会创建对象,必须通过send方法或者next方法对他进行预激活后,再进行后续的函数调用.一般来说协程需要使用send方法来驱动传递数据.

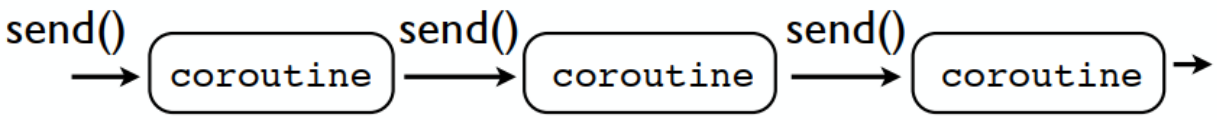
所有协程函数必须首先通过调用send(None)进行预激活,这将会使得执行推进到第一个yield表达式的地方,然后挂起.

```python
m=match('python')
print(m)
m.send(None)
```

我们需要注意的是,我们必须要先完成协程函数的预激活,将程序运行到第一个yield处挂起,然后才可以接受外部发送的数据.这里的send(None)语句就是在完成这一步预激活程序.如果我们不做预激活,而直接使用send(data),那么程序会返回一个TypeError,并告知不可以传入一个非None数据给未激活的生成器.由于这个预激活的操作是对于每个协程函数都需要调用的,因此我们可以使用装饰器来包装协程函数,将预激活操作封装到包装函数中,自动化预激活,这样就可以避免忘记预激活,减少了用户调用协程函数时的样板代码,提高接口的友好性.

```
def consumer(func):
    def wrapper(*args,**kwargs):
        cr=func(*args,**kwargs)
        cr.send(None)
        return cr
    return wrapper
@consumer
def match(pattern):
    print('Looking for %s' % pattern)
    while True:
        line =yield
        if pattern in line:
            print(line)
```

协程函数和生成器一样也可以用于组成管道.



我们只需要将协程函数链接在一起,并且通过send()操作将数据推过管道.这与生成器管道(Pull模型,依靠消费者拉取数据)不同,协程函数管道通常是Push模型,由生产者推送数据,数据通过send函数主动推进到下一个阶段,更适合事件驱动的场景.我们给出一些协程管道的代码实例,下面是一个典型的实时监控日志文件的模式

```
import time
def follow(filename,target):
    f=open(filename)
    f.seek(0,2) #Go to the end of the file
    # 可选，默认值为 0。给offset参数一个定义，表示要从哪个位置开始偏移；0代表从文件开头开始算起，1代表从当前位置开始算起，2代表从文件末尾算起
    while True:
        line=f.readline()
        if line!='':
            target.send(line)
        else:
            time.sleep(0.1)
@consumer
def printer():
    while True:
        line=yield
        print(line)
follow('../Data/stocklog.csv',printer())
```

follow函数在管道中充当了数据源的作用,其不断读取文件新行并且发送给目标,其中的printer是消费者.

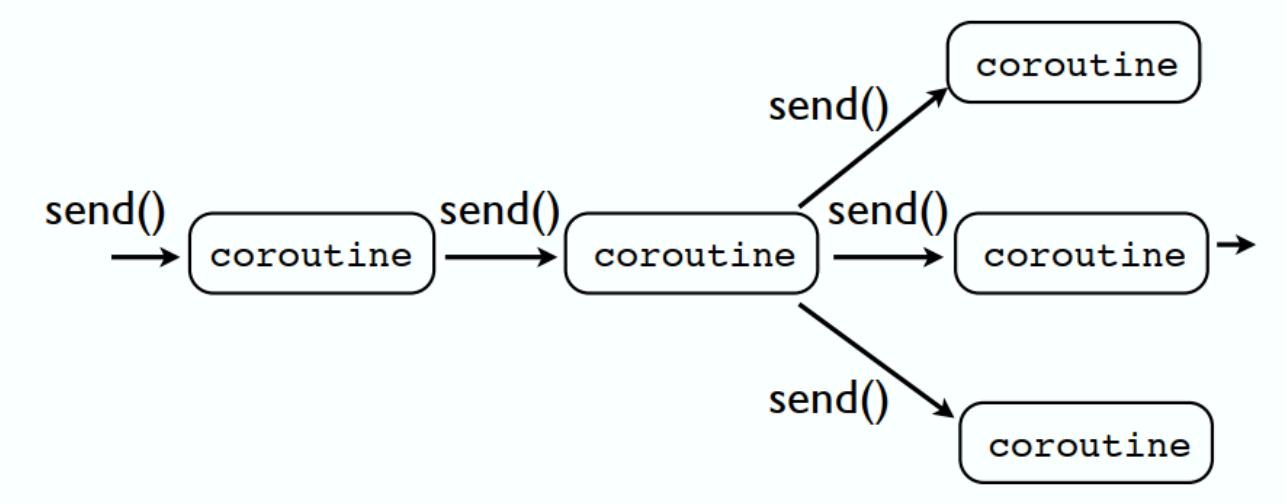
在上面的基础上,加上对数据的过滤操作.

```
@consumer
def match(pattern, target):
    while True:
        line = yield  # 接收一行
        if pattern in line:
            target.send(line)  # 发送到下一阶段
follow('../Data/stocklog.csv', match('AA', printer()))
```

这里形成了一个协作链,match函数接受数据并且过滤后,再发送给printer函数,嵌套调用建立了数据流向.

利用协程函数,我们可以实现对数据的扇出(fan out)

专



协程管道不仅局限于线性链,一个协程可以将数据发送给多个下游协程,当然也可以从多个上游协程处接受数据.

4 Ch8_Ex3 代码解释

这一部分的完整代码如下所示,所调用的包也会同步上传到Github,故我们只专注于讨论这个代码的调用逻辑和结构

```
from structure import Structure
from cofollow import consumer,follow
from tableformat import create_formatter
import csv
class Ticker(Structure):
    name = String()
```

```
    price =Float()
    date = String()
    time = String()
    change = Float()
    open = Float()
    high = Float()
    low = Float()
    volume = Integer()
@consumer
def to_csv(target):
    def producer():
        while True:
            yield line
    reader=csv.reader(producer())
    while True:
        line=yield
        target.send(next(reader))
@consumer
def create_ticker(target):
    while True:
        row = yield
        target.send(Ticker.from_row(row))
@consumer
def negchange(target):
    while True:
        record = yield
        if record.change < 0:
            target.send(record)
@consumer
def ticker(fmt, fields):
    formatter = create_formatter(fmt)
    formatter.headings(fields)
    while True:
        rec = yield
        row = [getattr(rec, name) for name in fields]
        formatter.row(row)
if __name__ == '__main__':
    follow('../Data/stocklog.csv',
           to_csv(
           create_ticker(
           negchange(
           ticker('text', ['name','price','change']))))))
```

首先第一步定义了一个名为Ticker的类,他继承自Structure.他规定了每一条股票数据应有的字段和类型,例如price是浮点数,volume是整数.这为后续对原始字符串的类型转换打下了基础.对于to_csv函数,他的装饰器的作用是自动执行一次next(coroutine),让协程函数启动并且停止在第一个yields处.这里的producer()是内部生成器,他被定义后不会运行,到后面的reader = csv.reader(producer()),此时producer()被调用返回一个生成器对象,并且csv.reader接受这个生成器作为他的数据源,注意此时producer产生的生成器还是没有执行的,等程序运行到line=yield挂起,直到外界数据源发送数据给line变量,然后producer会将这个line变量传递给csv.reader,然后在通过target.send()将解析好的列表传递给create_ticker.

create_ticker协程函数则是一个对象转换协程函数.他的作用是将原始列表按照类定义中的类型信息,把字符串转换为正确的Python类型.negchange函数的作用在这个处理中充当筛选器,如果某行数据不满足条件,就会在这里被丢弃,不会到达最后的打印环节.ticker函数则是数据管道的结束,他不会再往下发送数据,而是一句用户指定的fields提取数据并且格式化输出到控制台.管道的组装顺序,不同于生成器管道的由外层向内层请求数据,协程函数是从后向前定义的,并且是从外层向内层发送数据.

5 Generator Control Flow

生成器支持强制终止和异常处理.close方法表示强制终止生成器,throw方法则是抛出异常.生成器不仅仅是数据流,它还支持控制流.外部代码可以干预生成器的执行,比如强制关闭它(触发清理代码)或在内部抛出异常,这对资源管理非常重要.

调用close方法关闭生成器.

```
def genfunc():
    try:
        yield item
    except GeneratorExit:  # .close() 被调用
        # 执行清理（如果有）
        pass
        return
g = genfunc()  # 一个生成器
g.close()
```

如果生成器调用close方法关闭,它相当于会在yield处抛出一个GeneratorExit异常.close方法是确保资源释放的关键,如生成器打开了文件或者网络连接,则可以在调用close方法的时候抛出GeneratorExit异常的时候,在except块中关闭计算机关键资源,防止内存泄露.

使用throw(type[,val[,tb]])方法来抛出异常.

```
def genfunc():
    try:
        yield item
    except RuntimeError as e:  # 处理异常
        pass
g = genfunc()  # 一个生成器
g.throw(RuntimeError, "You're dead")
```

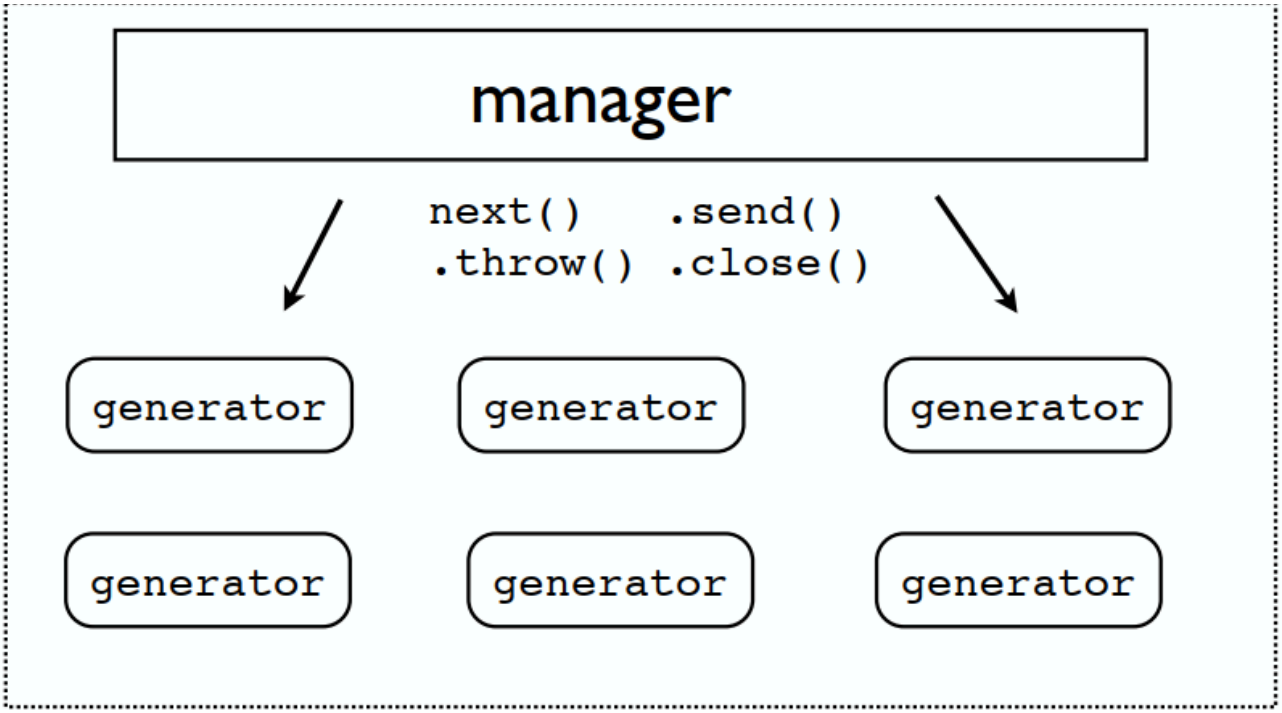
下我们简单介绍一下throw方法的传入的各个参数.type参数的作用是提供异常类型,其是必选参数,他指定了你想要抛出的异常类别.在执行的时候Python会检查这个类型并且将其抛出.val则表示异常值或异常实例,他是一个可选参数,用于代表异常的具体内容.如果val是一个实例,那么这个实例就是抛出的异常;如果是一个值,那么Python就会调用type(val)来创建一个异常实例;如果省略这个参数,Python则会尝试实例化一个没有参数的type.tb参数则表示回溯对象,这也是一个可选参数,一般来说平时不会调用.traceback对象用于定义异常发生的现场快照(调用栈信息).如果你想要在A处捕获异常,但在B处抛出它,并希望报错信息显示是在A处发生,可以将A处的__traceback__传递给这个参数.

throw()允许外部代码向生成器内部注入异常.这在协程管道中很有用,比如当某个阶段发生错误时,可以向下游传播错误信号,或者通知上游停止生产.需要注意的是异常并不会暂停生成器的存在,而只是允许yield将异常信息抛出.

6 Managed Generators

生成器函数并不能单独执行,他必须由其他东西驱动(如for循环或者send方法等).yield语句则表示一个抢占点,生成器会在yield处挂起,直到被外界调用或系统指示才会继续运行.其实这也是协作式多任务的基础,因为生成器需要调度器来分配计算资源,yield则是资源让出控制权的地方,这与操作系统线程的抢占式多任务不同.

我们引入一个manager模块来协调生成器的执行.



这里的管理器或者调度器就是协作式多任务的核心,他会维护一个生成器队列,轮流调用它们的next方法或者send方法,这使得多个生成器看起来像是在并发运行,实际上是在单线程中交替执行.他的典型应用场景为并发(如tasklets,greenlets等);Actors;事件模拟等.

我们给出如下的示例代码,

```
def countdown(n):
    while n>0:
        print('T-minus',n)
        yield
        n-=1
def countup(n):
    x=0
    while x<n:
        print('Up we go',x)
        yield
        x+=1
from collections import deque
tasks=deque([countdown(10),countdown(5),countup(20)])
def run():
    while tasks:
        t=tasks.popleft()
        try:
            t.send(None)
            tasks.append(t)
        except StopIteration:
            pass
run()
```

这里的yield没有产生值,也没有接受值,他只不过是自愿让出控制权的信号.任务执行一步,然后暂停让调度器运行其他的任务,这也就是协作式多任务的最简单的形式.他轮询任务队列,每次运行一个任务的一步.如果任务结束的时候就不再放回队列.这种模型非常适合I/O密集型任务,但CPU密集型任务上无法利用多核操作.

7 简单异步I/O服务器

```
from socket import *
from select import select
from collections import deque
```

导入socket模块,用于创建网络套接字.网络套接字也就是socket,是计算机网络中应用程序与网络协议栈之间进行通信的接口.他屏蔽了底层复杂的网络硬件细节,给程序员提供了一个简单的API.一个完整的socket链接由双方的IP,双方的端口以及通信协议(TCP/UDP)唯一确定.select则是操作系统提供的一个系统调用函数,他的核心作用是实现I/O多路复用.简单来说,select能让一个程序同时监视多个网络连接,当其中任何一个连接有数据可读或可写时,通知程序进行处理.导入deque则是创建双端队列,用于高效实现任务队列.

```
tasks=deque()
recv_wait={} # sock->task
send_wait={} # sock->task
```

tasks用于创建双端队列,用来存放当前可以立即执行的任务.recv_wait用于记录正在等待接收数据的任务,键是socket对象,值是等待该socket可读的任务;send_wait用于记录正在等待发送数据的任务,键是socket对象,值是等待该socket可写的任务.这三个容器构建了调度器的任务状态.

```
def run():
    while any([tasks,recv_wait,send_wait]):
        while not tasks:
            can_recv,can_send,_=select(recv_wait,send_wait,[])
            for s in can_recv:
                tasks.append(recv_wait.pop(s))
            for s in can_send:
                tasks.append(send_wait.pop(s))
        task=tasks.popleft()
        try:
            reason,resource=task.send(None)
            if reason == 'recv':
                recv_wait[resource] = task
            elif reason == 'send':
                send_wait[resource] = task
            else:
                raise RuntimeError('Unknown reason %r' % reason)
```

```
        except StopIteration:
            print('Task done')
```

run函数中的while any循环表示是任务队列tasks非空或者还有任务在等待I/O(在recv_wait和send_wait),那么循环就继续.while not task循环则是表示如果当前没有可以立即执行的任务,也就是所有任务都在等待I/O;select程序在此阻塞,他会监视recv_wait中的socket(查看谁有数据可读)和send_wait中的socket(看谁可以写入数据).can_recv是select返回的可读socket列表,can_send是select返回的可写socket列表.for循环的作用则是遍历准备好的可读/可写socket列表,从等待字典中取出对应的任务并从字典中删除,将其引入到可立即执行任务队列,所以如果I/O就绪,唤醒对应任务,让其回到任务队列.task.popleft()则是从任务队列的左端取出一个任务执行.try语法块的task.send(None)则是启动它,使其重新获得CPU资源.后续的任务函数的yield会返回值('recv'和socket)被解包赋值给reason和resource.他会告知调度器暂停的原因为reason,需要关注的资源是resource.按照暂停的原因,将任务挂起在不同的容器中,将控制权重新交还给run循环.当生成器函数执行完毕(没有更多yield且函数返回),会抛出StopIteration异常,打印任务完成,然后run循环继续处理下一个任务.

```
def tcp_server(address, handler):
    sock = socket(AF_INET, SOCK_STREAM)
    sock.setsockopt(SOL_SOCKET, SO_REUSEADDR, 1)
    sock.bind(address)
    sock.listen(5)
    while True:
        yield 'recv', sock
        client, addr = sock.accept()
        tasks.append(handler(client, addr))
```

socket(AF_INET, SOCK_STREAM)的作用是用来制作一个网络通信的工具(套接字对象),其中所用的参数AF_INET指定使用的是IPv4地址协议,如果使用AF_INET6则表示支持IPv6地址协议;SOCK_STREAM指定使用TCP协议,面向连接的,可靠的传输,SOCK_DREAM表示使用UDP协议.setsockopt(SOL_SOCKET,SO_REUSEADDR,1)的作用是允许端口被立即重用.参数的作用是SOL_SOCKET表示设置的选项层级的Socket层级;SO_REUSEADDR表示允许地址复用,1表示开启.他的作用是如果关闭服务器程序后,操作系统会让端口进入TIME_WAIT状态,以确保网络中残留的旧数据包消失.如果没有这一行,立即重启服务器,那么就会报地址占用的错误.bind(address)非套接字分配一个具体的地址,address是一个元组(host,port),如果套接字没有固定的地址,客户端不知道去哪里连接.listen(5)表示将套接字从主动模式切换为被动监听模式,准备接受客户端连接,5表示backlog队列长度,当服务器忙不过来时,操作系统内核可以帮忙暂时排队存放多少个连接请求.如果此时第6个客户端在此时尝试连接,而服务器还没有处理完前5个,第6个连接可能会被直接拒绝.如果不写的话,那么套接字只是一个普通的连接点,无法调用accept()方法来接受新连接.此时sock就会变成一个标准的服务器监听套接字.

while循环是一个死循环,表示服务器程序需要一直运行,不能处理完一个连接就退出,他必须时刻等待新的客户端连接.但是虽然他是一个死循环,但yield的存在是可以主动让出控制权,所以其他任务有机会运行.yield的作用是暂停当前任务,并且向调度器提供自己的I/O兴趣,run函数会将当前的函数放入到recv_wait字典,键是sock,程序执行权会回到run函数.如果run函数的select发现sock可读,那么重新唤醒tcp_server任务才会继续向下执行.一般来说accept需要等待连接,但是此时因为select已经可确定了sock上有了链接请求,所以他会马上执行,不需要等待,他返回的client是一个新的socket对象,他不是监听的socket,而是专门用于和这个客户端通信的socket,addr则是客户端的IP地址和端口.append操作则是创建并调度一个新的任务,handler则会根据传入的socket对象和addr地址创建一个任务.上面的并发原理其实是服务器主程序不断等待连接,同时不断创建新的客户任务,处理数据收发.

```
def echo_handler(client, address):
    print('Connection from', address)
    while True:
        yield 'recv', client
        data = client.recv(1000)
        if not data:
            break
        yield 'send', client
        client.send(b'GOT:' + data)
    print('Connection closed')
```

while True也是一个死循环,因为一个TCP链接建立以后,客户端可能会发送多条消息,而不是只调用一次就会断开,这个循环则确保服务器能持续处理该客户端的多次请求,直到客户端主动断开.yield 'recv',client这个语句会暂停任务,并向外申请可读需求.如果进入了下一步,那么表示client这个socket已经开始接受数据了,recv(1000)的作用则是从socket读取最多1000个字节的数据.需要注意的是实际上recv函数是可能会发生阻塞的,但是我们在此调用的时候已经默认了存在一个socket读取的数据,所以不会阻塞,可以立刻拿到数据.后面的if判断则是判断客户端是否断开连接;data是空字节,表示客户端已经关闭了连接(发送了FIN码),则结束循环;data如果有内容就继续运行.yield 'send',client这个语句会暂停任务,并向外申请可写需求.实际上send也是存在阻塞的可能,操作系统的发送缓冲区会满,这样的话send方法就会发生阻塞,所以需要防止因发送过快导致整个服务器线程卡住.send操作是将收到的数据前面加上GOT,这里需要注意的是网络通信的本质上是二进制,所以计算机网络底层传输的只能是0和1,也就是字节,所以添加的字符串也应该是字节字符串.

完整代码如下

```
from socket import *
from select import select
from collections import deque
tasks=deque()
recv_wait={} # sock->task
```

```python
send_wait={} # sock->task
def run():
    while any([tasks,recv_wait,send_wait]):
        while not tasks:
            can_recv,can_send,_=select(recv_wait,send_wait,[])
            for s in can_recv:
                tasks.append(recv_wait.pop(s))
            for s in can_send:
                tasks.append(send_wait.pop(s))
        task=tasks.popleft()
        try:
            reason,resource=task.send(None)
            if reason == 'recv':
                recv_wait[resource] = task
            elif reason == 'send':
                send_wait[resource] = task
            else:
                raise RuntimeError('Unknown reason %r' % reason)
        except StopIteration:
            print('Task done')
def tcp_server(address, handler):
    sock = socket(AF_INET, SOCK_STREAM)
    sock.setsockopt(SOL_SOCKET, SO_REUSEADDR, 1)
    sock.bind(address)
    sock.listen(5)
    while True:
        yield 'recv', sock
        client, addr = sock.accept()
        tasks.append(handler(client, addr))
def echo_handler(client, address):
    print('Connection from', address)
    while True:
        yield 'recv', client
        data = client.recv(1000)
        if not data:
            break
        yield 'send', client
        client.send(b'GOT:' + data)
    print('Connection closed')
if __name__ == '__main__':
    tasks.append(tcp_server(('',25000), echo_handler))
    run()
```

这里的(",25000)中"代表0.0.0.0,意思是监听本机所有网卡(无论是局域网IP,公网IP还是本地回环);25000代表端口号,客户端必须链接这个端口才能找到服务器.

8 Other Genarator

如果我们考虑如下生成器函数,

```
def countdown(n):
    while n>0:
        yield n
        n-=1
def countup(n):
    x=0
    while x<n:
        yield x
        x+=1
def up_and_down(n):
    countup(n)
    countdown(n)
```

需要注意的是up_and_down函数就算被调用他也不会产生结果,因为他只会获得生成器对象,但不会将其消耗.当然我们可以将函数改写为如下结构,

```
def up_and_down(n):
    for x in countup(n):
        yield x
    for x in countdown(n):
        yield x
```

我们需要自己驱动生成器,也就是需要使用for循环手动控制每个生成器,但是如果函数结构比较复杂,那么手动管理就会十分复杂,因此我们可以引入yield from语法来将生成器的驱动交给Python解释器驱动,无论什么外部代码运行生成器,都会自行处理,

```
def up_and_down(n):
    yield from countup(n)
    yield from countdown(n)
```

async的协程我们已在前面提及到了,如感兴趣可以查阅协程,所以不再赘述.

Modules and Packages

1 Modules

模块化和包机制是Python管理复杂性的核心手段,也是代码复用的基础.每个源文件都是一个模块,import用于加载并执行一个模块.我们在到如模块的同时也到入了其定义的命名空间.

```
import foo
foo.grok()
# 相当于如下的调用方式
foo.__dict__['grok']()
```

所以module的命名空间本质上就是一个字典,在主程序中可以用module.__dict__访问,这也意味着你可以在运行时动态修改模块的内容.模块文件中其实还有一些特殊变量定义,如__file__表示源文件名称,__name__表示模块名称,__doc__表示模块文档字符串.因为主程序其实也可以认为是一个模块文件,只不过他的模块名是main.上面提到的__name__属性是区分源文件是直接运行还是被导入执行的关键,如果文件是直接运行的,那么name就是main,而如果被导入的,那么name就是模块文件名,这就是编写可复用模块的标准模式.

我们在此简要介绍一下模块导入的流程,以如下的伪代码解释,

```
import types
def import_module(name):
    # 定位模块文件并获取源代码
    filename=find_module(name)
    code=open(filename).read()
    # 创建enclosing模块对象
    mod=types.ModuleType(name)
    # 运行源代码
    exec(code,mod.__dict__,mod.__dict__)
    return mod
```

源代码在模块字典中通过exec执行,存储的内容就是执行完exec的结果.所以导入的机制为寻找模块文件,创建一个新的模块对象,在该对象的命名空间的字典中执行文件代码.所以模块文件的全局变量实际上会变成模块对象的属性.import语句将模块对象插入到导入模块的文件所使用的字典.其流程是先加载和执行模块,然后创建一个模块变量,指向这个模块对象.每个模块都只会加载一次,重复import就只会返回之前加载过的以前模块的引用.

```
import sys
print(list(sys.modules))
```

sys.modules指的是所有已经加载模块的字典.模块是一个单例模式,如果在不同地方多次导入,它们指向的是内存中同一个模块对象,这也意味着模块级的全局变量在所有调用文件之间是共享的.我们在此简单的引入模块的单例模式,其伪代码如下所示,

```
import types
import sys
def import_module(name):
    # 检查模块缓存
    if name in sys.modules:
        return sys.modules[name]
    filename=find_module(name)
    code=open(filename).read()
    mod=types.ModuleType(name)
    exec(code,mod.__dict__,mod.__dict__)
    return mod
```

我们通过如下代码选择性导入模块中的一部分变量,适用于频繁适用的变量名,

```
from module import var
```

但这不会改变导入的工作方式,模块依旧会被加载和执行,然后会在程序中复制变量名,大体如下所示,

```
grok=sys.modules['foo'].grok
```

我们可以用如下代码获取模块中的所有代码并将它们放入调用文件的命名空间,

```
from module import *
```

但他只适用于不以_开头的变量,所以我们可以用_name来定义模块中不想被导入的值.需要提醒的是import *被视作一个编程的坏习惯,其污染了当前命名空间,使代码来源不清晰,而以下划线开头的名称被视为私有,不会被通配符导入.

在上面我们提及了模块不可以被重复加载,但是实际上我们可以调用importlib使得模块可重新加载.

```
import importlib
importlib.reload(foo)
```

2 Packages

Python库的标准做法是组织为位于顶层包名下的分层模块集,如下所示,

```
packagename
packagename.foo
packagename.bar
packagename.utils
packagename.utils.spam
packagename.utils.grok
packagename.parsers
packagename.parsers.xml
packagename.parsers.json
...
```

这样的层次命名结构可以避免在大型程序中的命名冲突.为了创建Python中的包,应该先创建模块库的层次结构,将文件组织在具有所需结构的目录中,

```
packagename/
            __init__.py
            foo.py
            bar.py
            utils/
                  __init__.py
                  spam.py
                  grok.py
            parsers/
                  __init__.py
                  xml.py
                  json.py
```

这里我们可以看见创建包在每个目录文件中创建__init__.py,这个文件可以是空的,但是必须要存在.他的存在会告知Python解释器该目录应该视作一个包.一旦我们设置了init.py,那我们就可以使用import语句导入包.

```
import packagename.foo
import packagename.parsers.xml
from packagename.parsers import xml
```

几乎所有内容的工作方式与以前一样,只是import语句会具备多层级别.这里我们需要注意到子模块的相对导入会失效,也就是

```
# spam/
#     foo.py
#     __init__.py
#     bar.py
# bar.py
import foo # Fails
```

所以这可以解决了顶层包和子模块之间的名称冲突,

```
# bar.py
import os
```

这里import导入方式是绝对的,从顶层开始往下查找的.在包内部,直接使用import语句会被是做绝对导入,他会在sys.path查找系统级的包,而不是在同级目录下查找,这样可以避免包内模块意外覆盖标准模块.我们需要用绝对导入的方式来获取包内模块,它需要调用顶层包的名称,

```
from spam import foo
```

但这样如果我们修改包结构的话,那么这样的绝对导入就需要修改整体的代码,因此Python又提供了包相对导入的方法,其中.表示当前包,..表示父包.

```
# bar.py
from . import foo # Imports ./foo.py
from .foo import name # Load a specific name
from .grok import blah # Imports ./grok/blah.py
```

相对导入使得包内部重构更容易,但只能在包内使用,不可以用作顶层脚本的直接导入.包内也会定义一些有用的特殊变量,如__package__为封装包的名称,__path__则是子模块的搜索路径,

```
import xml
print(xml.__package__)
print(xml.__path__)
```

3 __init__ File

__init__.py的主要用途是将多个源文件缝合到一个顶层导入中.__init__.py是包的入口,不仅标记目录为包,还可以执行初始化代码,或者导出子模块的内容,简化用户的段数路径.

```
# foo.py
class Foo:pass
#bar.py
class Bar:pass
#__init__.py
from .foo import Foo
from .bar import Bar
# main.py
import spam
f = spam.Foo()
b = spam.Bar()
```

用户会只看到一个统一的顶层包,将跨子模块的拆分隐藏起来.这是一个优秀的API设计模式,内部实现可以分散在多个文件中以方便维护,但对外只暴露一个简洁的接口.不仅如此,我们可以在子模块中定义__all__列表用于控制from module import *传导的变量属性,可以隐藏一些子模块的内部实现,

```
# foo.py
__all__ = ['Foo']
class Foo(object): pass
def func: pass
```

这样的话,如果在__init__.py中导入foo中的全部变量,其实只有类Foo,而没有func函数.所以__all__列表明确声明了模块的公共API.它不仅控制import *的行为,也是给阅读代码的人看的文档,表明哪些内容是稳定的接口,哪些是内部实现.

4 Main Module

上面的导入模块中,我们需要十分小心包内的循环导入,应该小心的遵循控制流,因此定义的顺序十分重要.例如A导入B,B导入A,模块执行完成前就会尝试访问一个未定义的内容,那就会报错.一般来说都是因为导入放在模块的顶部,如果需要改正这一问题一般是将导入语句放在不同的地方,避免循环依赖.将导入语句移动到函数内部或者类定义之后,可以出现延迟导入,避免执行时候的冲突.最好的解决方案通常是重构代码,将共享内容提取到一个单独的模块中,打破循环依赖.

将指定模块作为主程序运行,这样可以将支持的脚本或者应用程序封装在包里面.

```
python -m module
```

-m标志告诉Python查找模块并执行它,即使该模块在包内部.我们也可以为包提供一个主入口程序点__main__.py,使得包目录可以执行,这样的话我们可以直接使用python -m package,他会自动执行__main__.py,这样可以让包像前面的程序一样简单运行.